



Vue 3 完整学习指南

这是一份全面、系统、图文并茂的 Vue 3 框架深度学习资料，融合基础原理、高级特性与工程实践，旨在帮助开发者从入门到精通。



目录结构

- [第一部分：核心基础](#)
- [第二部分：高级特性](#)
- [第三部分：工程实践](#)
- [第四部分：性能优化](#)
- [第五部分：面试题汇总](#)

第一部分：核心基础



Vue 是什么？

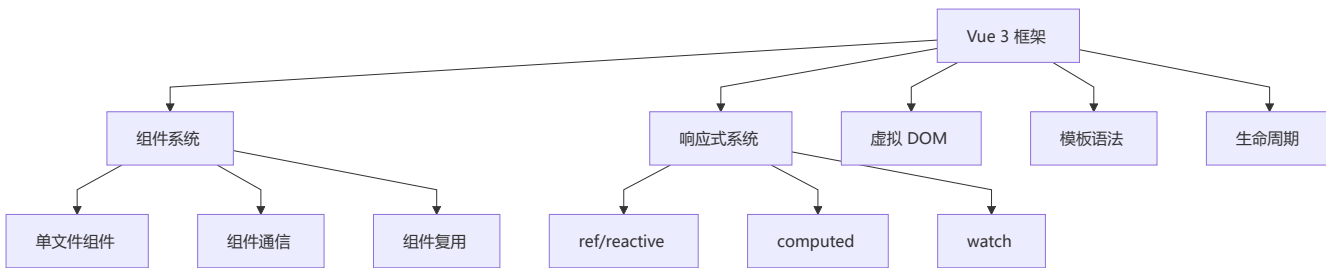


核心定义

Vue 是一个渐进式 JavaScript 框架，用于构建用户界面。它以**易学易用**、**高性能**和**灵活的组件化**而闻名，是介于 React 的自由度和 Angular 的完整性之间的完美平衡。



Vue 的核心角色



框架对比

特性	Vue	React	Angular
学习曲线	● 平缓	● 中等	● 陡峭
灵活性	✓ 高	✓ 极高	⚠ 受限
性能	✓ 优秀	✓ 优秀	✓ 优秀
生态	✓ 完整	✓ 最庞大	✓ 完整
中文资源	✓ 丰富	⚠ 中等	⚠ 有限

2 Vue 3 新特性详解

🌟 重要特性速览

```
Vue 3 (2020+)
├─ Composition API (组合式 API)
├─ Proxy 响应式系统
├─ Fragment & Teleport
├─ Suspense (异步组件)
├─ 更快的性能
├─ TypeScript 优化
└─ 新的全局 API
```

🔗 Composition API 详解

问题背景

Vue 2 选项式 API 在处理复杂组件时，相关逻辑被分散在不同选项中，难以维护：

```
// ❌ 问题：逻辑分散
export default {
  data() {
    return { count: 0, name: '' };
  },
  computed: {
    doubledCount() { return this.count * 2; }
  },
  methods: {
    increment() { this.count++; }
  },
  watch: {
    name(newVal) { console.log(newVal); }
  }
};
```

解决方案：Composition API

```
// ✅ 解决：逻辑聚合
import { ref, computed, watch } from 'vue';

export default {
  setup() {
    const count = ref(0);
    const name = ref('');

    const doubledCount = computed(() => count.value * 2);
    const increment = () => { count.value++; };

    watch(name, (newVal) => {
      console.log(newVal);
    });
  }
};
```

```
    return { count, name, doubledCount, increment };  
  }  
};
```

改进点:

- ☒ 逻辑聚合，易于阅读和维护
- ☒ 强大的代码复用能力
- ☒ 更好的 TypeScript 支持
- ☒ 灵活的代码组织

⚡ Proxy 响应式系统

Vue 2 vs Vue 3

vue 2: Object.defineProperty

缺点:

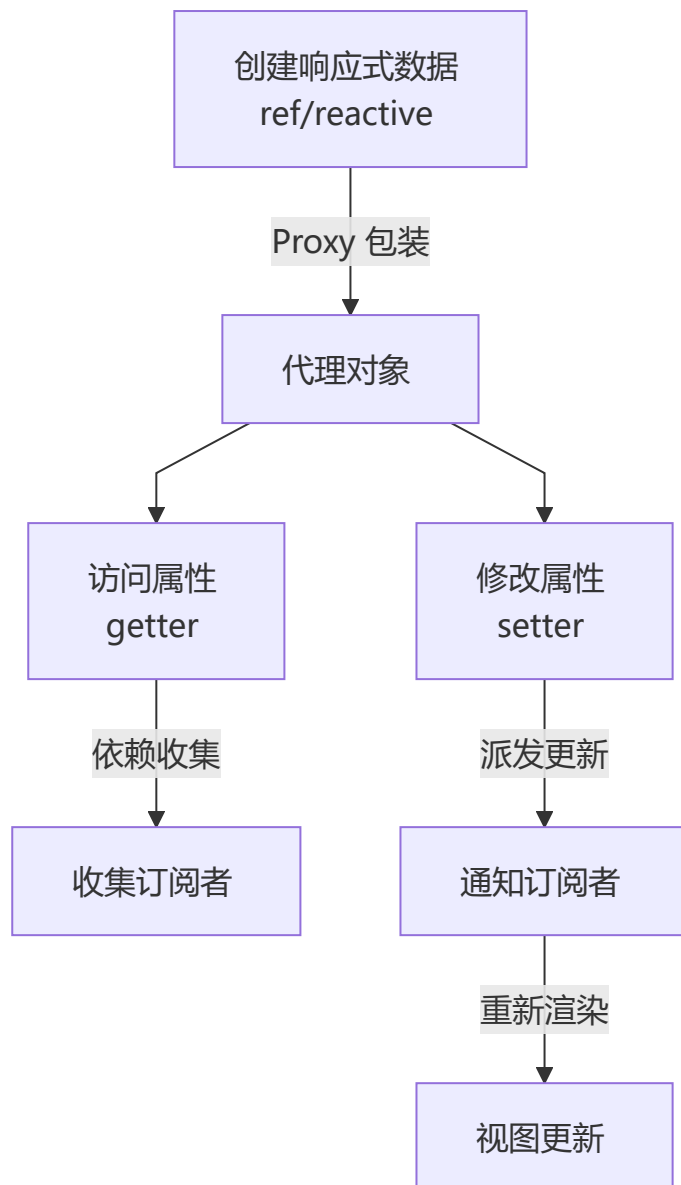
- ✗ 无法检测属性新增/删除
- ✗ 数组索引变化无法检测
- ✗ 需要 `$set` 进行强制更新

Vue 3: Proxy

优点:

- ☒ 全面响应式（任何变化）
- ☒ 直观的 API 设计
- ☒ 更好的 TypeScript 支持
- ☒ 惰性处理（按需响应）

工作原理图



ref vs reactive

```
// ref - 适用于任何类型
const count = ref(0);
const user = ref({ name: 'John' });

console.log(count.value); // 0
// 模板中自动解包: <div>{{ count }}</div>

// reactive - 仅适用于对象
const state = reactive({
  count: 0,
  user: { name: 'John' }
});

console.log(state.count); // 0
// 模板中也无需 .value: <div>{{ state.count }}</div>
```

```
// ⚠ reactive 重新赋值会丢失响应性
state = { count: 5 }; // ❌ 错误
state.count = 5;      // ✅ 正确
```

3 响应式原理深入

📌 Vue 的基本原理

Vue 通过 `Object.defineProperty` (Vue 2) / `Proxy` (Vue 3) 实现数据响应式，核心三要素是 **Observer** (观察者)、**Dep** (依赖收集器)、**Watcher** (订阅者)，三者协作驱动视图更新。

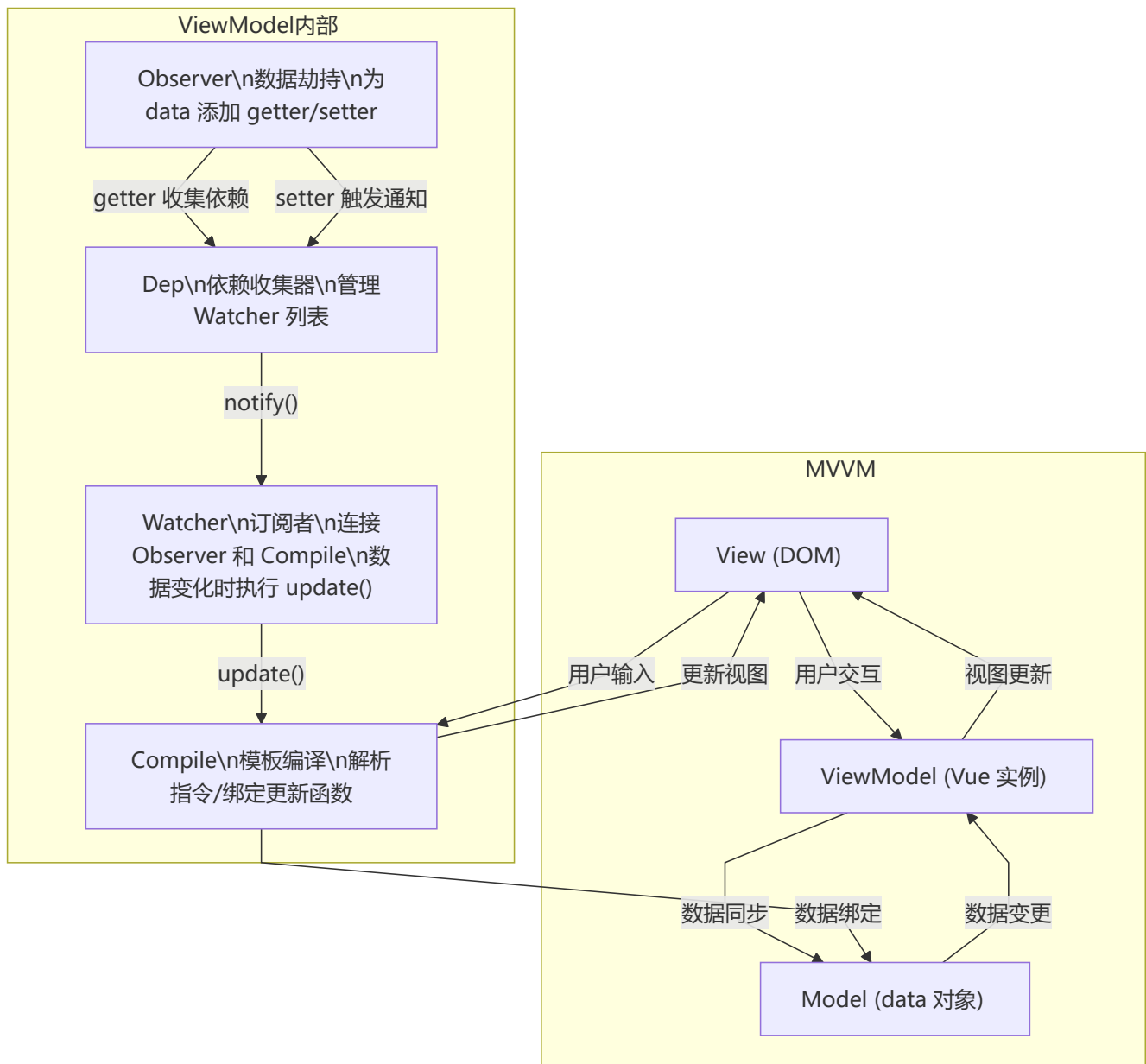
当一个 Vue 实例创建时，Vue 会遍历 `data` 中的属性，用 `Object.defineProperty` (Vue 3 使用 `Proxy`) 将它们转为 `getter/setter`，并在内部追踪相关依赖。当属性被访问时收集依赖，被修改时通知变化。

核心三要素：

- **Observer**：递归遍历 `data`，为每个属性添加 `getter/setter`
- **Dep**：每个响应式属性对应一个 `Dep`，管理订阅它的 `Watcher`
- **Watcher**：组件渲染 `Watcher`、计算属性 `Watcher`、用户 `Watcher`

🔄 双向数据绑定的原理

Vue 采用**数据劫持 + 发布者-订阅者模式**，通过 `Object.defineProperty()` 劫持各属性的 `setter/getter`，在数据变动时发布消息给订阅者，触发相应监听回调。

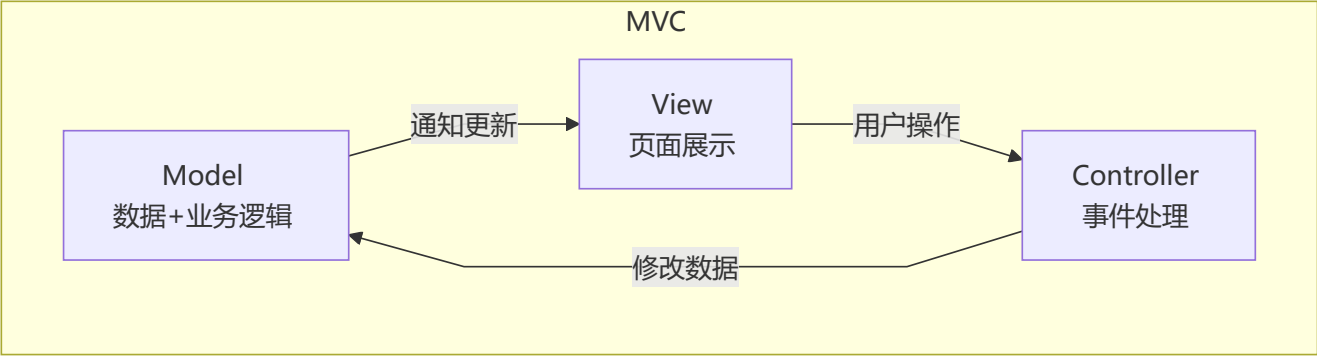
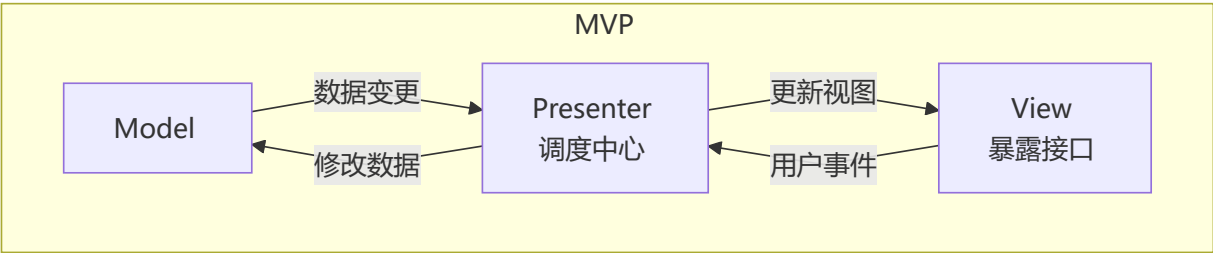
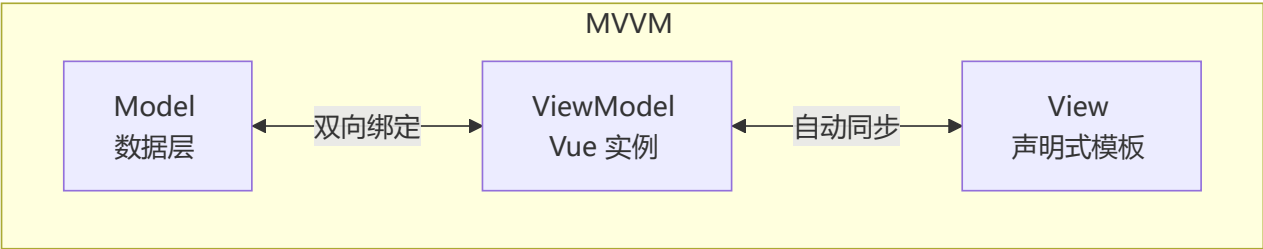


核心流程四步：

1. **Observer**: 递归遍历 data 对象，为所有属性添加 getter/setter
2. **Compile**: 解析模板指令，将模板中的变量替换为数据，为每个指令对应的节点绑定更新函数
3. **Watcher**: 作为 Observer 和 Compile 之间的通信桥梁，属性变动时触发 Compile 回调
4. **MVVM**: 整合 Observer、Compile 和 Watcher 三者，实现双向绑定



MVVM / MVC / MVP 架构对比



特性	MVC	MVP	MVVM
View 与 Model 关系	观察者模式，直接耦合	完全解耦	完全解耦
通信中枢	Controller	Presenter	ViewModel
数据流	单向	单向	双向
DOM 操作	需要手动操作	需要手动操作	自动完成



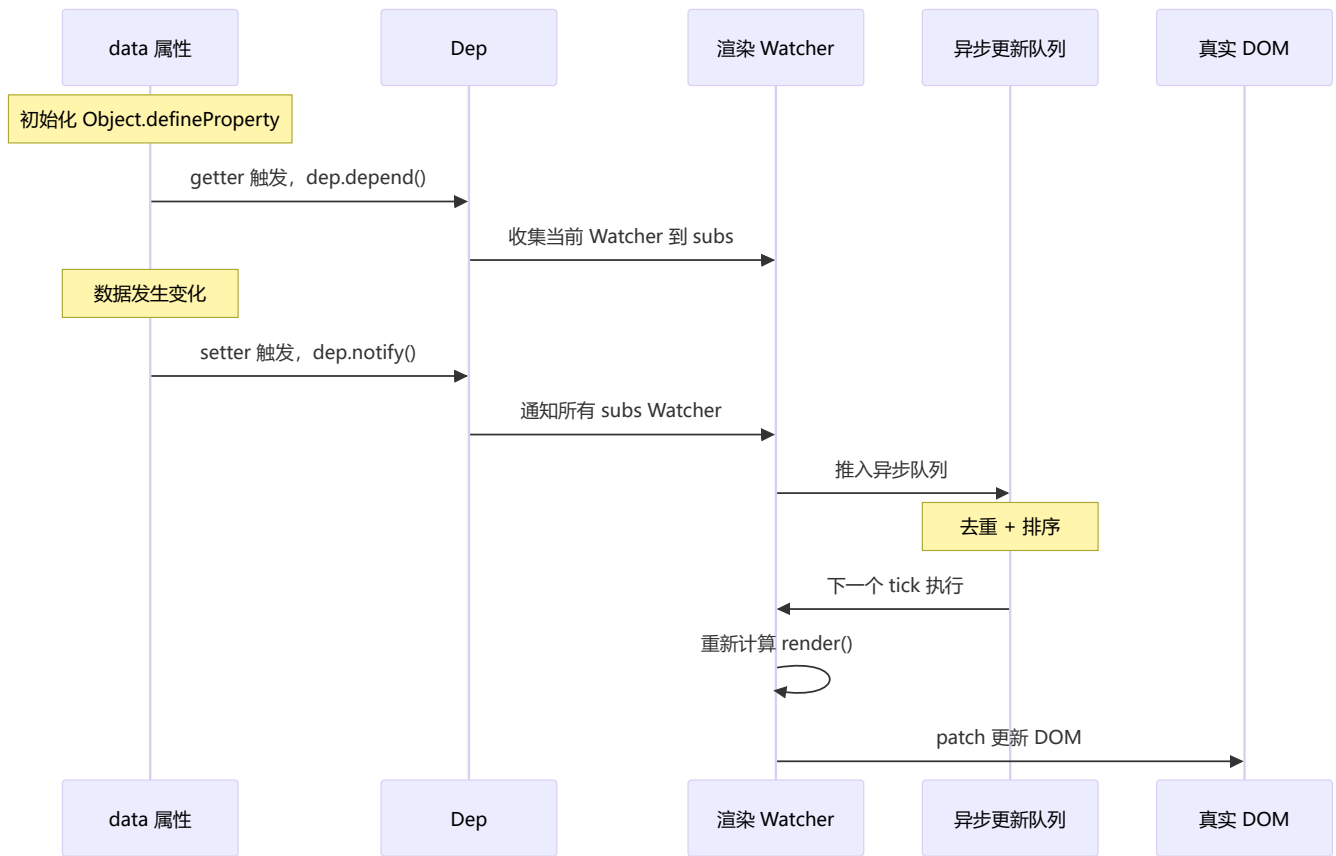
依赖收集原理



角色	作用
Dep	依赖收集器，subs 数组存储所有订阅它的 Watcher
watcher	订阅者，有 update()、get()、addDep() 等方法

角色	作用
<code>Dep.target</code>	静态属性，全局唯一，指向当前正在计算的 Watcher

响应式数据更新流程



`Object.defineProperty()` 的缺陷与 Proxy 的改进

问题	说明	Vue 2 的变通方案
无法监听数组下标赋值	<code>arr[0] = newVal</code> 不会触发 setter	重写数组的 7 个方法
无法监听数组长度变化	<code>arr.length = 0</code> 无响应	同上
无法监听对象新增属性	<code>obj.newKey = val</code> 无响应	使用 <code>vue.set()</code>
无法监听对象删除属性	<code>delete obj.key</code> 无响应	使用 <code>vue.delete()</code>
需要递归遍历	初始化时就要深度遍历所有属性	一次性开销

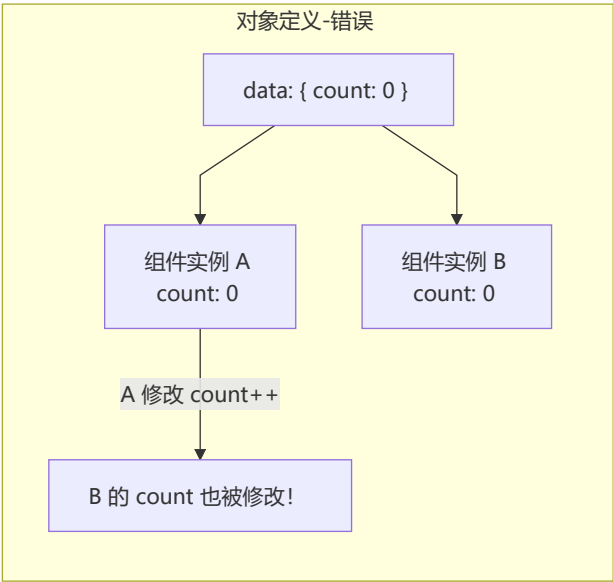
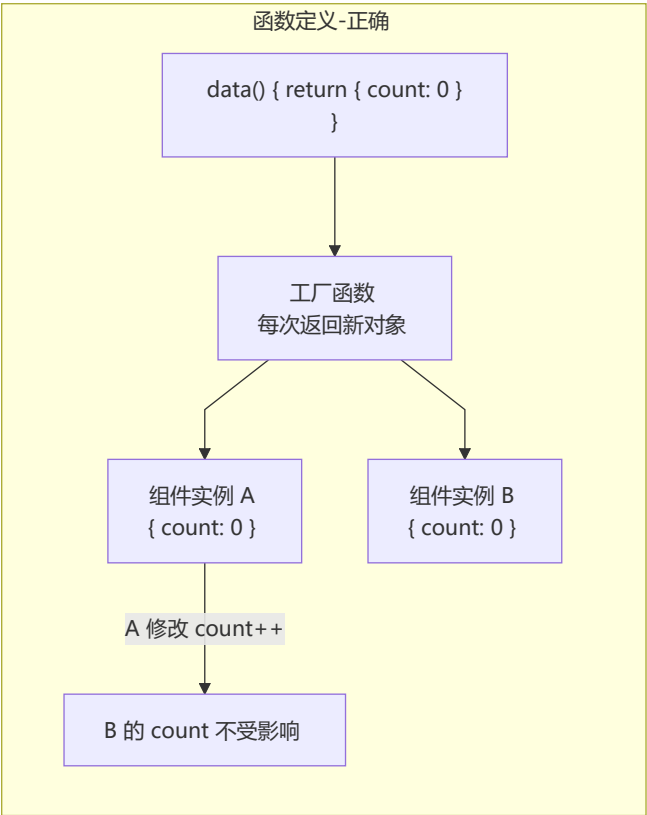
Proxy 的优势 (Vue 3) :

能力	<code>Object.defineProperty</code> (Vue 2)	Proxy (Vue 3)
拦截方式	逐个定义属性	代理整个对象
对象新增属性	❌ 需 <code>Vue.set()</code>	✅ 自动检测

能力	Object.defineProperty (Vue 2)	Proxy (Vue 3)
对象删除属性	❌ 需 Vue.delete()	✅ 自动检测
数组下标修改	❌ 需重写方法	✅ 原生支持
数组长度变更	❌ 不可检测	✅ 原生支持
Map/Set	❌ 不支持	✅ 原生支持
初始化性能	递归遍历所有属性	懒代理（访问时才代理深层对象）

? data 为什么是函数

组件 data 必须是函数，每次创建组件实例时返回新的数据对象，避免多个实例共享同一引用导致数据污染。



⚡ \$nextTick 原理

`$nextTick` 利用微任务/宏任务机制在 DOM 更新后执行回调，实现数据变更与 DOM 操作的时序协调。

Event Loop 中的 nextTick

数据变更

将 Watcher 加入异步队列

flushSchedulerQueue
(去重、排序、执行)

尝试

异步任务优先级

Promise.then (微任务)

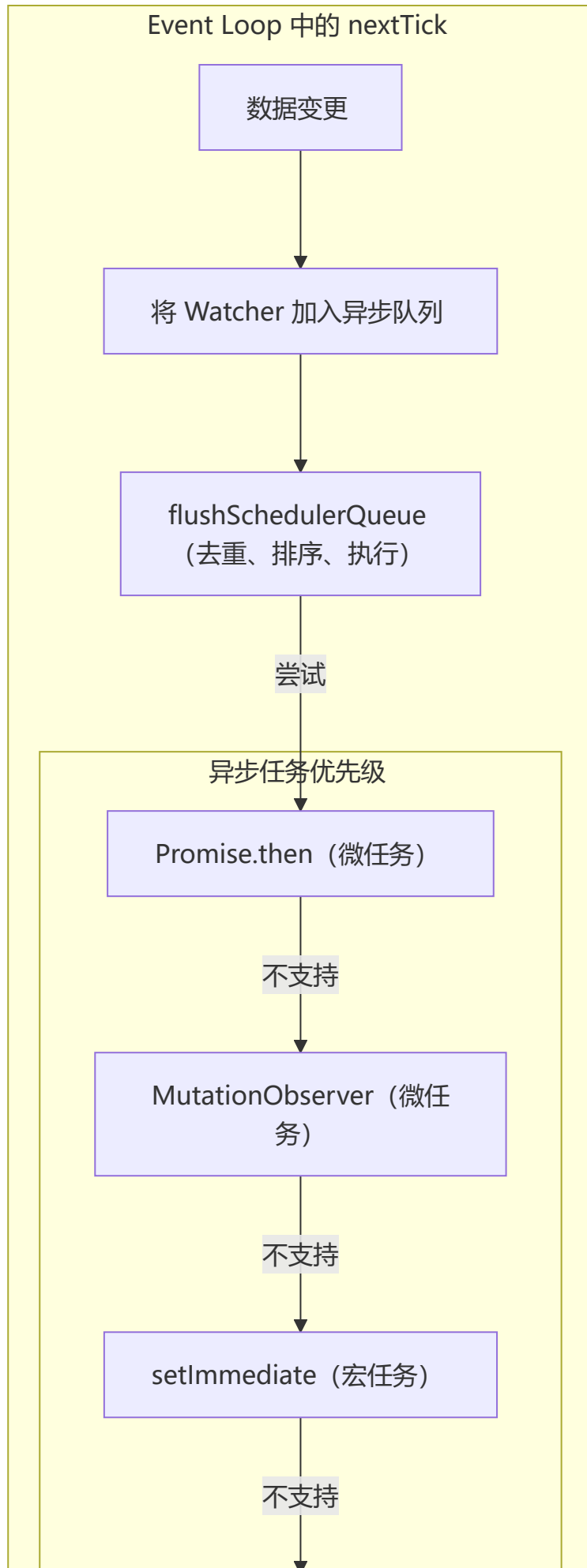
不支持

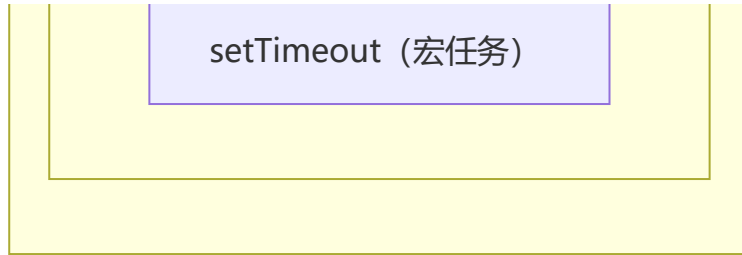
MutationObserver (微任务)

不支持

setImmediate (宏任务)

不支持





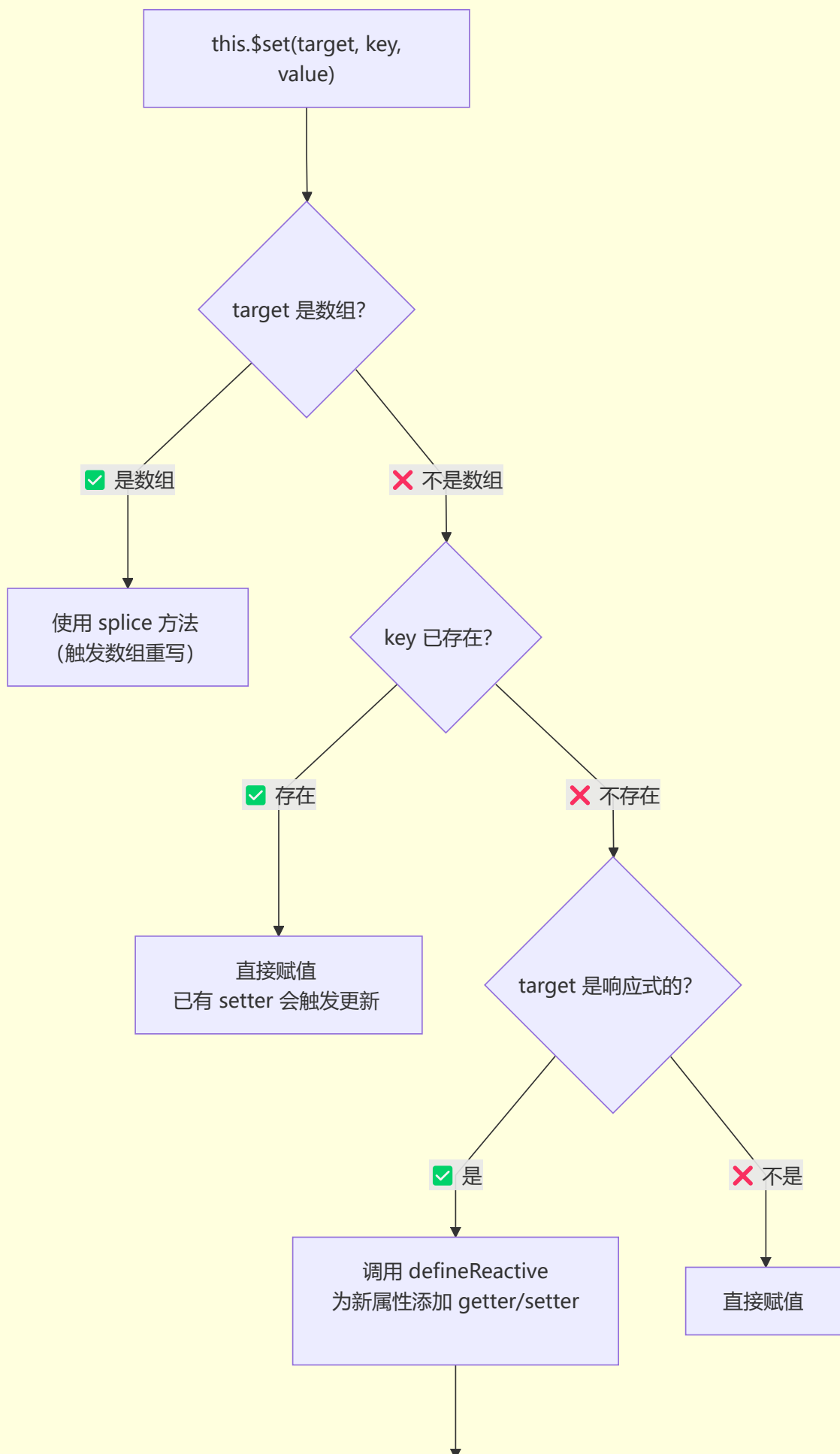
异步更新的好处：

1. **去重**：多次对同一个属性赋值，只会触发一次更新
2. **合并**：同一 tick 内的多次数据变更合并为一次 DOM 操作
3. **性能**：避免频繁的 DOM 重排重绘

💡 `$set` 原理 (Vue 2)

`$set` 用于向响应式对象添加新属性或修改数组元素，内部通过 `defineReactive` 为新属性添加响应式并手动触发 `dep.notify()`。

\$set 实现逻辑



手动通知 `dep.notify()`

🔧 Vue 数组方法重写 (Vue 2)

Vue 2 通过重写数组的 7 个方法实现数组响应式: `push` / `pop` / `shift` / `unshift` / `splice` / `sort` / `reverse`。

执行流程

arr.push(newVal)

执行原生 push

observeArray(inserted)
监听新增元素

dep.notify()
通知 Watcher 更新

原型链拦截

Array.prototype

Vue 重写

arrayMethods (新对象)

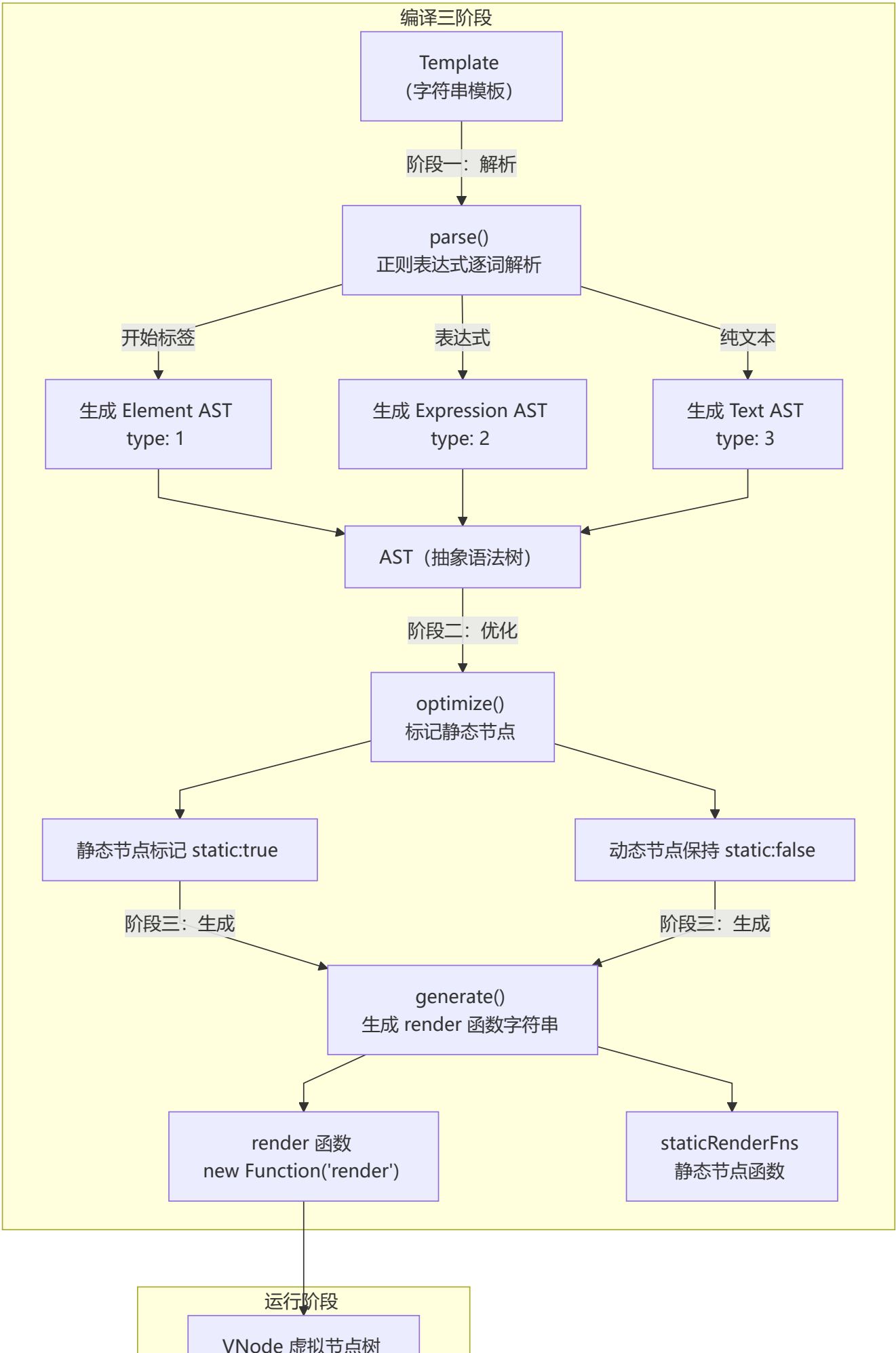
通过 **proto** 覆盖

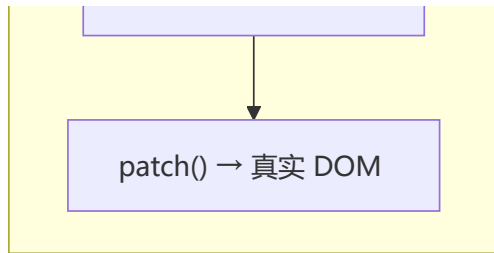
数组实例

```
arr.proto = arrayMethods
```

Template → Render 编译过程

模板编译分三阶段：`parse`（解析为 AST）→ `optimize`（标记静态节点）→ `generate`（生成 render 函数）。





阶段	函数	输入	输出	核心工作
解析	<code>parse()</code>	template 字符串	AST	正则表达式解析标签、指令、属性
优化	<code>optimize()</code>	AST	带标记的 AST	标记静态节点，跳过后续 diff
生成	<code>generate()</code>	AST	render 函数字符串	拼接成可执行的 JS 代码

4 组合式 API 完全指南

🔗 核心函数详解

`ref` - 单个值的响应式

```
import { ref } from 'vue';

const count = ref(0);
const name = ref('Alice');

console.log(count.value); // 0
count.value = 5;
// 模板中自动解包: {{ count }} -> 5
```

`computed` - 计算属性

```
import { ref, computed } from 'vue';

const count = ref(1);
const multiplier = ref(2);

// 只读计算属性
const doubled = computed(() => count.value * multiplier.value);

// 可写计算属性
const userFullName = computed({
  get: () => `${firstName.value} ${lastName.value}`,
  set: (newValue) => {
    const [first, last] = newValue.split(' ');
    firstName.value = first;
    lastName.value = last;
  }
});
```

watch - 监听变化

```
import { ref, watch } from 'vue';

const count = ref(0);
const user = ref({ name: 'Alice', age: 30 });

// 监听单个 ref
watch(count, (newVal, oldVal) => {
  console.log(`Count changed from ${oldVal} to ${newVal}`);
});

// 监听对象（深度监听）
watch(user, (newVal, oldVal) => {
  console.log('User changed:', newVal);
}, { deep: true });

// 监听多个源
watch([count, user], ([newCount, newUser]) => {
  console.log('Count or User changed!');
});

// 取消监听
const stopWatching = watch(count, () => {});
stopWatching();
```

watchEffect - 自动依赖追踪

```
import { ref, watchEffect } from 'vue';

const count = ref(0);

// 自动追踪依赖
watchEffect(() => {
  console.log(`Count is now: ${count.value}`);
});

// 获取停止函数
const stop = watchEffect(() => {});
stop(); // 停止监听
```

生命周期钩子

```
import {
  onBeforeMount, onMounted,
  onBeforeUpdate, onUpdated,
  onBeforeUnmount, onUnmounted
} from 'vue';

onMounted(() => {
  console.log('组件已挂载');
```

```
const timer = setInterval(() => {}, 1000);

onUnmounted(() => {
  clearInterval(timer);
});
});
```

Computed vs Watch vs Methods

特性	Computed	Watch	Methods
缓存	✅ 依赖不变不重新计算	❌ 每次变化都触发	❌ 每次调用都执行
异步支持	❌ 不支持	✅ 支持	✅ 支持
适用场景	派生数据、模板简化	数据变化后执行副作用	事件处理、主动调用
返回值	必须返回一个值	不要求返回值	不要求返回值
模板可用	✅ {{ computedProp }}	❌ 不可直接用	✅ {{ method() }}

🔗 完整组合式 API 示例

```
<script setup lang="ts">
import { ref, computed, watch, onMounted, onUnmounted } from 'vue';

const count = ref(0);
const multiplier = ref(2);

const doubled = computed(() => count.value * multiplier.value);
const countStatus = computed(() => {
  if (count.value < 0) return '负数';
  if (count.value === 0) return '零';
  return '正数';
});

watch(count, (newVal) => {
  console.log(`Count changed to ${newVal}`);
});

let timer: number;
onMounted(() => {
  timer = setInterval(() => {}, 1000);
});
onUnmounted(() => {
  clearInterval(timer);
});

const increment = () => count.value++;
const decrement = () => count.value--;
const reset = () => count.value = 0;
</script>
```

```

<template>
  <div>
    <h2>计数器</h2>
    <p>Count: {{ count }} ({{ countStatus }})</p>
    <p>Doubled: {{ doubled }}</p>
    <button @click="increment">+</button>
    <button @click="decrement">-</button>
    <button @click="reset">重置</button>
  </div>
</template>

```

5 模板语法完全参考

指令和绑定

```

<!-- 插值表达式 -->
<div>{{ message }}</div>
<div>{{ count + 1 }}</div>

<!-- 属性绑定 -->

<button :disabled="isDisabled">按钮</button>
<div :class="{ active: isActive }">动态类</div>
<div :style="{ color: activeColor }">动态样式</div>

<!-- 事件绑定 -->
<button @click="handleClick">点击</button>
<input @keyup.enter="submitForm" />

<!-- 双向绑定 -->
<input v-model="message" />
<input v-model.number="age" />
<input v-model.trim="name" />

<!-- 条件渲染 -->
<div v-if="visible">显示</div>
<div v-else-if="loading">加载中...</div>
<div v-else>隐藏</div>
<div v-show="visible">始终存在于 DOM</div>

<!-- 列表渲染 -->
<ul>
  <li v-for="item in items" :key="item.id">{{ item.name }}</li>
</ul>

<!-- 特殊属性 -->
<component :is="currentComponent" />
<KeepAlive>
  <component :is="currentComponent" />
</KeepAlive>

```

v-model 修饰符

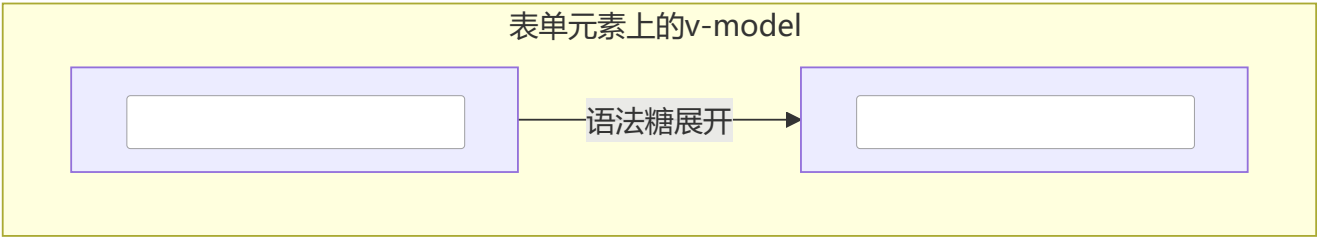
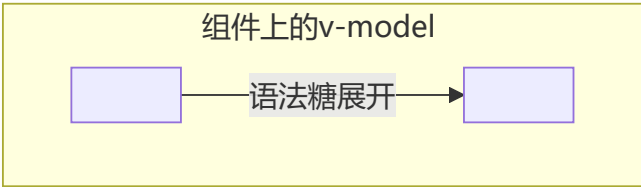
修饰符	作用	示例
<code>.number</code>	自动转为数字	<code>v-model.number="age"</code>
<code>.trim</code>	去除首尾空格	<code>v-model.trim="name"</code>
<code>.lazy</code>	改为 change 事件触发	<code>v-model.lazy="message"</code>

动态指令参数 (Vue 3.3+)

```
<div :[attributeName]="value">动态属性</div>
<button @[eventName]="handler">动态事件</button>
```

 v-model 实现原理 (语法糖)

v-model 本质是 `:value + @input` 的语法糖，表单元素监听原生事件，自定义组件默认利用 `value` prop 和 `input` 事件。

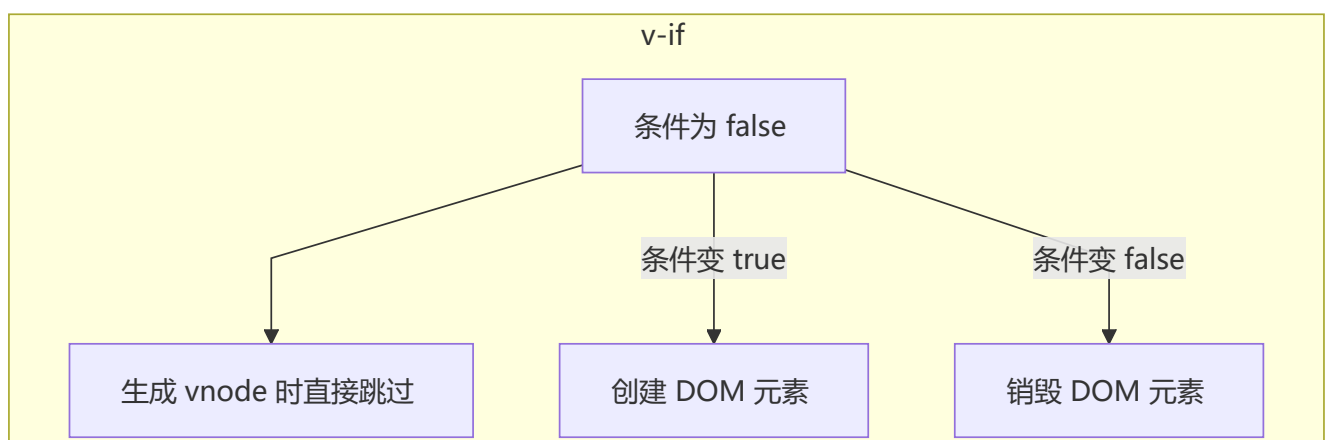
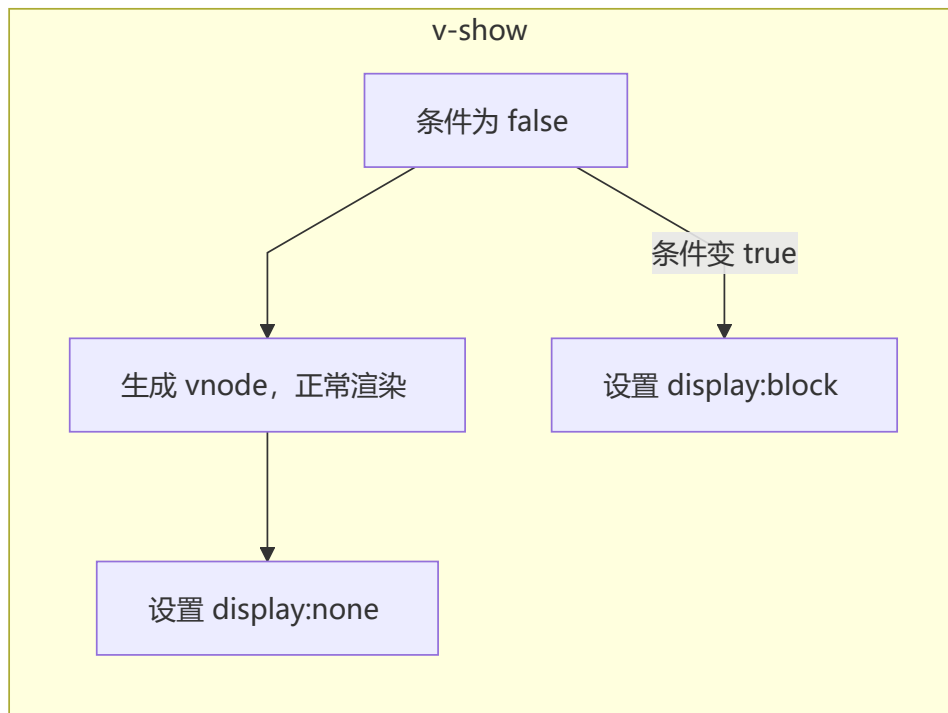
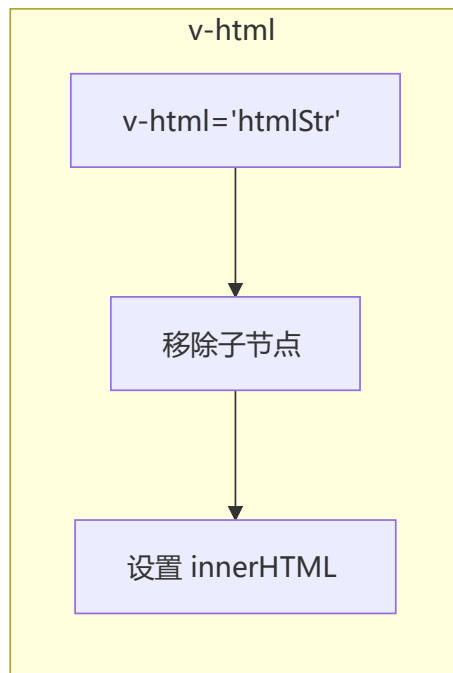


```
// v-model 语法糖本质：绑定 value 与监听 input 事件
v-model = v-bind:value + v-on:input
```

 v-if vs v-show vs v-html 原理

指令	渲染方式	切换成本	初始渲染	适用场景
<code>v-if</code>	条件渲染，销毁/创建 DOM	高 (重建)	条件为 false 时不创建	不频繁切换
<code>v-show</code>	始终渲染，切换 <code>display</code>	低 (CSS 切换)	始终创建 DOM	频繁切换

指令	渲染方式	切换成本	初始渲染	适用场景
<code>v-html</code>	直接设置 <code>innerHTML</code>	—	—	渲染 HTML（警惕 XSS）



✖ Slot 插槽机制

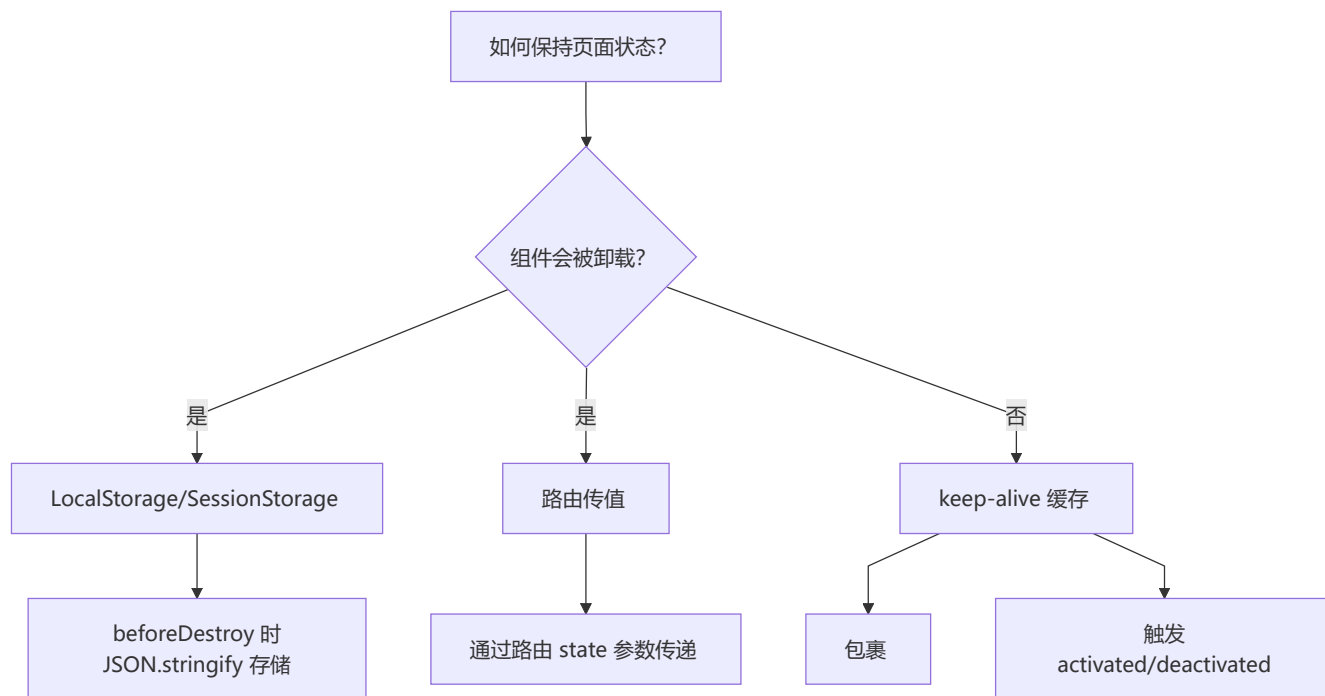
插槽是组件内容分发的核心机制，分为默认插槽（匿名）、具名插槽（name 属性）、作用域插槽（子传数据给父）。



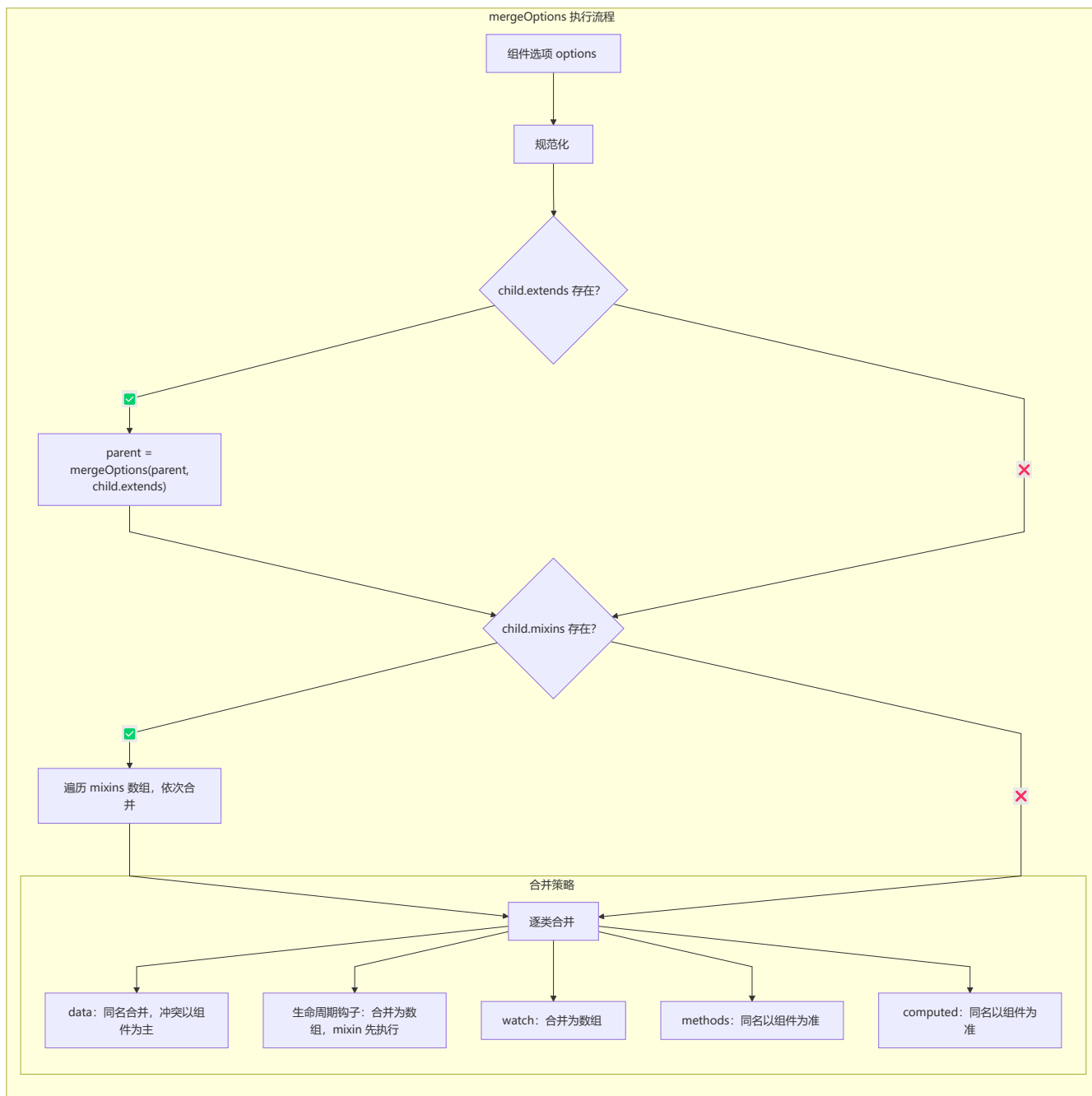
插槽分类:

- **默认插槽**（匿名插槽）： `<slot>`，组件内只有一个
- **具名插槽**： `<slot name="xxx">`，一个组件可有多个
- **作用域插槽**：子组件通过 `slot` 标签属性传递数据给父组件

保持页面状态的方案



6 mixin / extends 合并策略

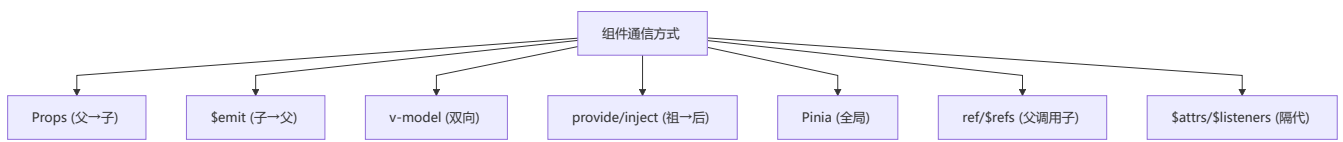


合并优先级: 组件自身 > mixins (后传入优先) > extends > 全局 mixin

第二部分：高级特性

7 组件通信完全指南

通信方式对比



方式	适用场景	方向
props / \$emit	父子组件	父→子 / 子→父
ref / \$refs	父子组件	父调用子实例
\$parent / \$children	父子组件	任意方向
provide / inject	祖孙组件	祖先→后代
\$attrs / \$listeners	隔代组件	祖先→深层后代
eventBus	任意组件	双向
Pinia / Vuex	复杂状态管理	全局

Props - 父向子传递

```
<!-- 父组件 -->
<script setup lang="ts">
import Child from './Child.vue';
const message = ref('Hello from parent');
</script>

<template>
  <Child :message="message" :user="user" :count="123" />
</template>

<!-- 子组件 -->
<script setup lang="ts">
interface User { name: string; age: number; }

const props = defineProps<{
  message: string;
  user: User;
  count?: number;
}>();

const propsWithDefaults = withDefaults(
  defineProps<Props>(),
```

```

    { count: 0 }
  );
</script>

<template>
  <p>{{ props.message }}</p>
</template>

```

📌 \$emit - 子向父传递

```

<!-- 子组件 -->
<script setup lang="ts">
const emit = defineEmits<{
  (e: 'increment', value: number): void;
  (e: 'update-name', name: string): void;
}>();

const handleClick = () => emit('increment', 1);
</script>

<template>
  <button @click="handleClick">增加</button>
</template>

<!-- 父组件 -->
<template>
  <Child @increment="(val) => count += val" />
</template>

```

📌 v-model - 双向绑定

```

<!-- 子组件 -->
<script setup lang="ts">
const props = defineProps<{ modelValue: string }>();
const emit = defineEmits<{ (e: 'update:modelValue', value: string): void }>();

const handleInput = (e: Event) => {
  emit('update:modelValue', (e.target as HTMLInputElement).value);
};
</script>

<template>
  <input :value="modelValue" @input="handleInput" />
</template>

<!-- 父组件 -->
<script setup lang="ts">
const message = ref('Hello');
</script>

<template>

```

```
<Child v-model="message" />
<!-- 等价于 <Child :modelValue="message" @update:modelValue="message = $event" /> -->
</template>
```

provide/inject - 跨层传递

```
<!-- 祖先组件 -->
<script setup lang="ts">
import { provide, ref } from 'vue';

const theme = ref('light');
provide('theme', theme);
provide('toggleTheme', () => {
  theme.value = theme.value === 'light' ? 'dark' : 'light';
});
</script>

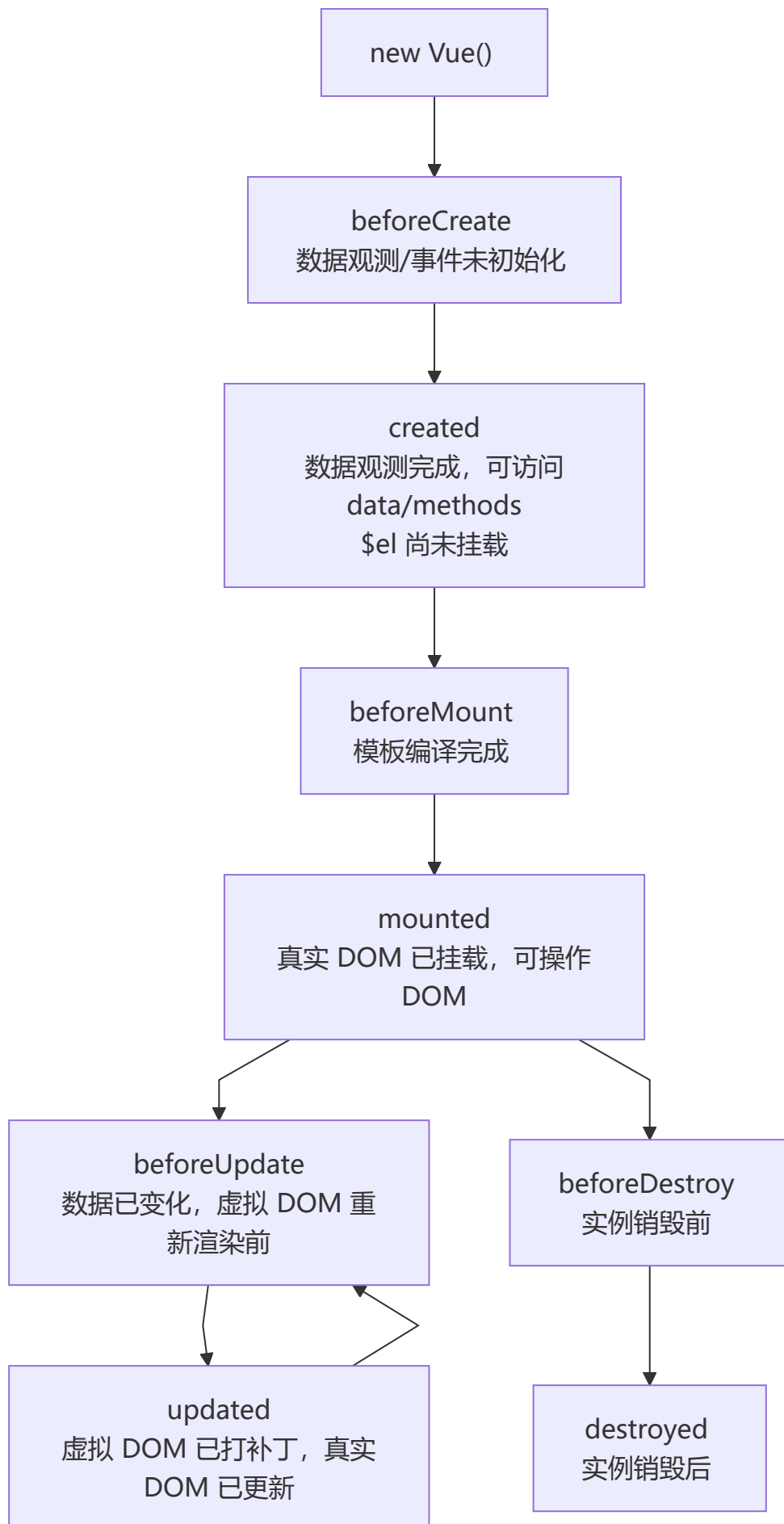
<!-- 后代组件 -->
<script setup lang="ts">
import { inject } from 'vue';

const theme = inject<'light' | 'dark'>('theme', 'light');
const toggleTheme = inject<() => void>('toggleTheme');
</script>

<template>
  <div :class="theme">
    <p>当前主题: {{ theme }}</p>
    <button @click="toggleTheme">切换主题</button>
  </div>
</template>
```

8 生命周期详解

完整生命周期流程图

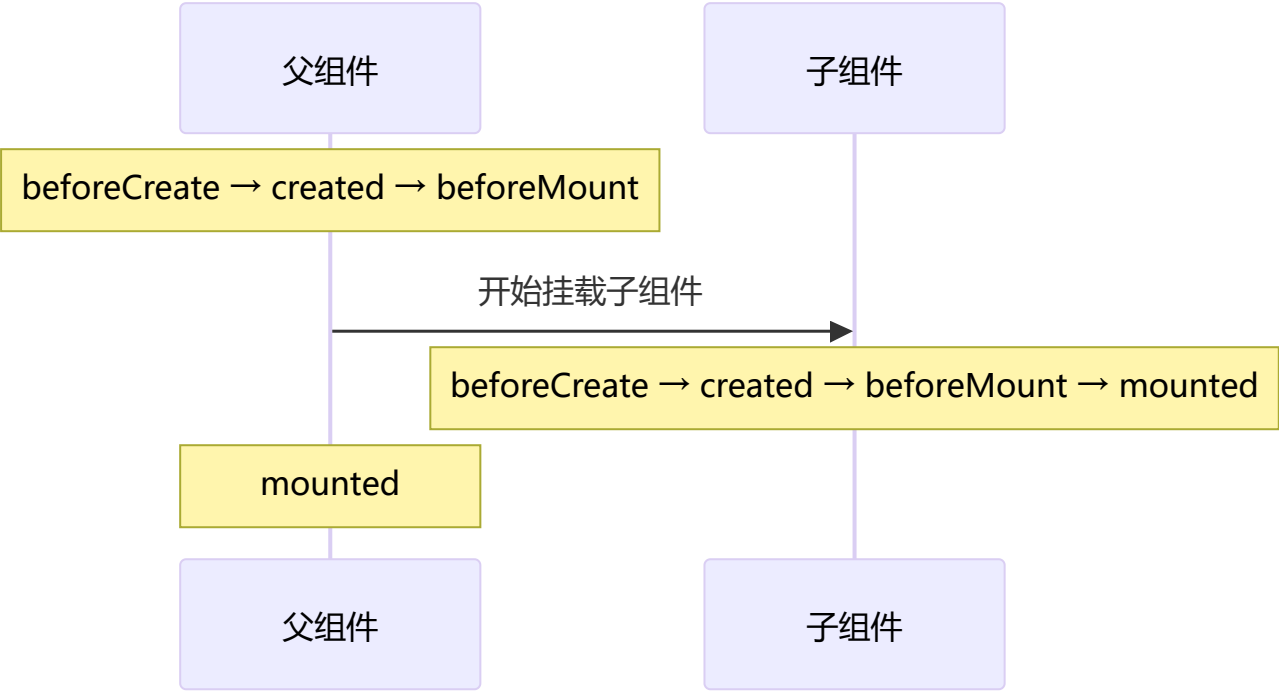


Vue 3 组合式 API 生命周期钩子:

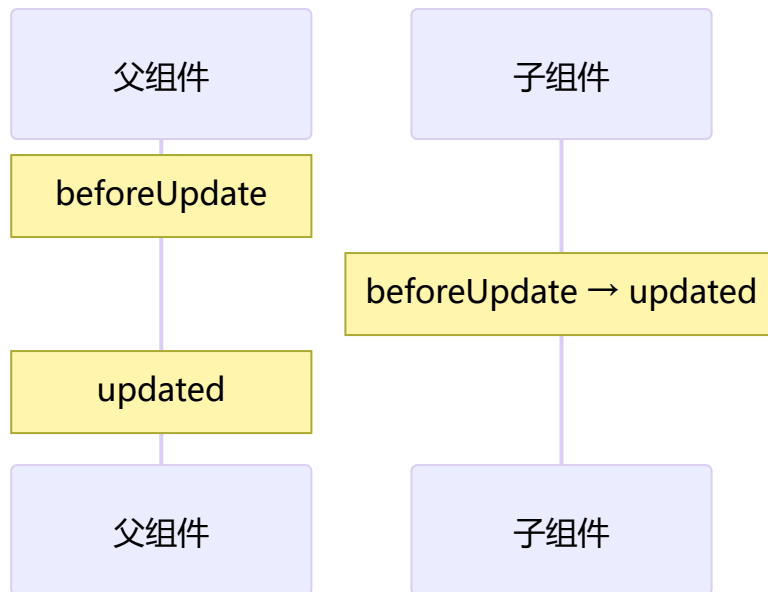
Vue 2 选项式	Vue 3 组合式	执行时机
<code>beforeCreate</code>	<code>setup()</code>	组件实例化后，Props 解析完成后执行
<code>created</code>	<code>setup()</code>	同上
<code>beforeMount</code>	<code>onBeforeMount</code>	组件挂载到 DOM 前
<code>mounted</code>	<code>onMounted</code>	组件挂载到 DOM 后
<code>beforeUpdate</code>	<code>onBeforeUpdate</code>	响应式数据更新，DOM 尚未更新
<code>updated</code>	<code>onUpdated</code>	DOM 更新完毕
<code>beforeDestroy</code>	<code>onBeforeUnmount</code>	组件卸载前
<code>destroyed</code>	<code>onUnmounted</code>	组件卸载后（清理定时器、事件监听）
<code>activated</code>	<code>onActivated</code>	被 <code><keepAlive></code> 缓存的组件激活时
<code>deactivated</code>	<code>onDeactivated</code>	被 <code><keepAlive></code> 缓存的组件停用时
<code>errorCaptured</code>	<code>onErrorCaptured</code>	捕获子组件树中的错误

📌 父子组件生命周期顺序

挂载阶段：



更新阶段：

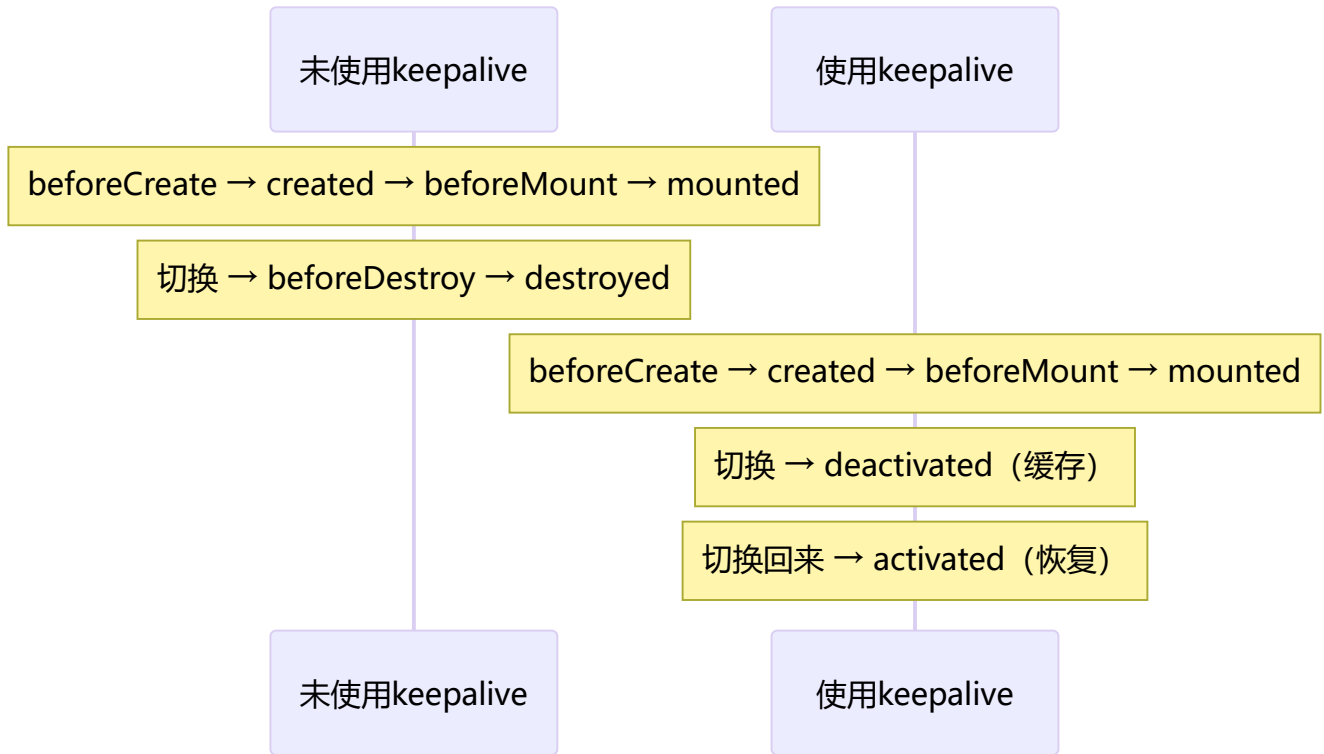


销毁阶段:



关键提示: 遵循"父组件创建 → 子组件创建 → 子组件挂载 → 父组件挂载"的顺序。更新阶段父 beforeUpdate 先触发，子更新完成后再触发父 updated。

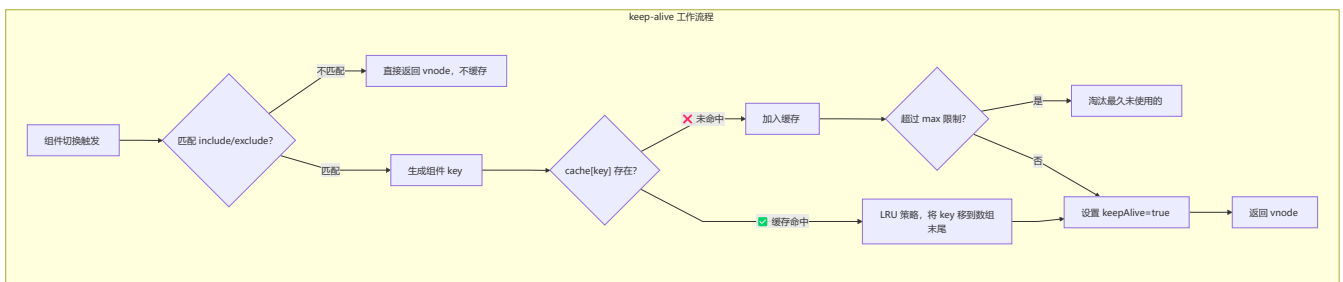
keep-alive 专属生命周期



9 模板与渲染进阶

keep-alive 实现原理

`keep-alive` 是抽象组件，通过 LRU 缓存策略缓存组件 `vnode`，新增 `activated` / `deactivated` 生命周期，命中缓存时不触发 `created` / `mounted`。

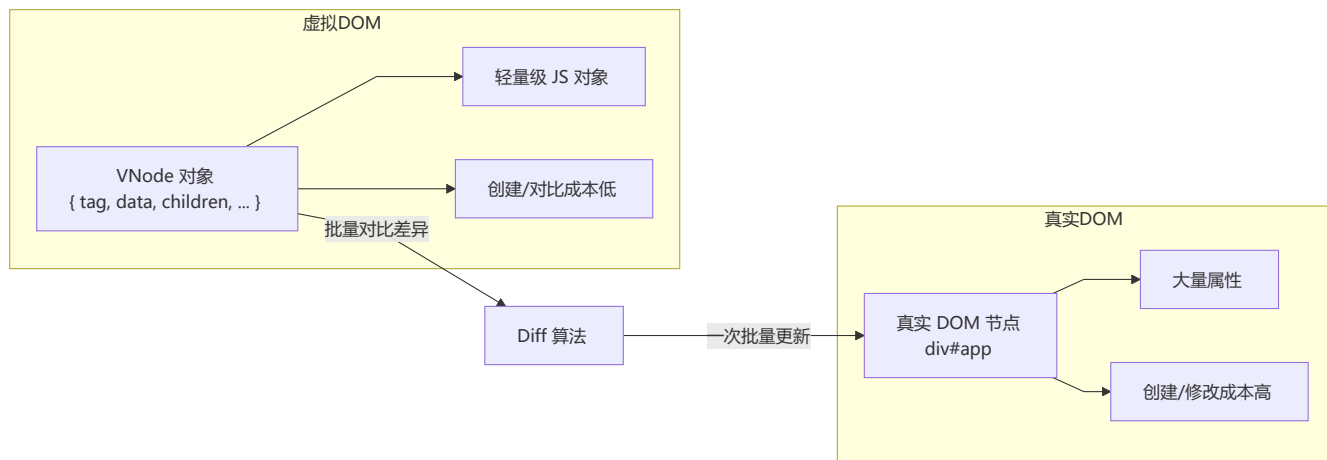


核心要点：

- `keep-alive` 是一个抽象组件（`abstract: true`），自身不渲染 DOM 元素
- 通过 `cache` 对象缓存组件的 `vnode` 实例
- 采用 **LRU (Least Recently Used)** 缓存淘汰策略
- 缓存命中时不会执行 `created`、`mounted` 等钩子，而是直接插入缓存的 DOM
- 新增 `activated` 和 `deactivated` 两个生命周期钩子

⚡ 虚拟 DOM 与 Diff 算法

虚拟 DOM 概述

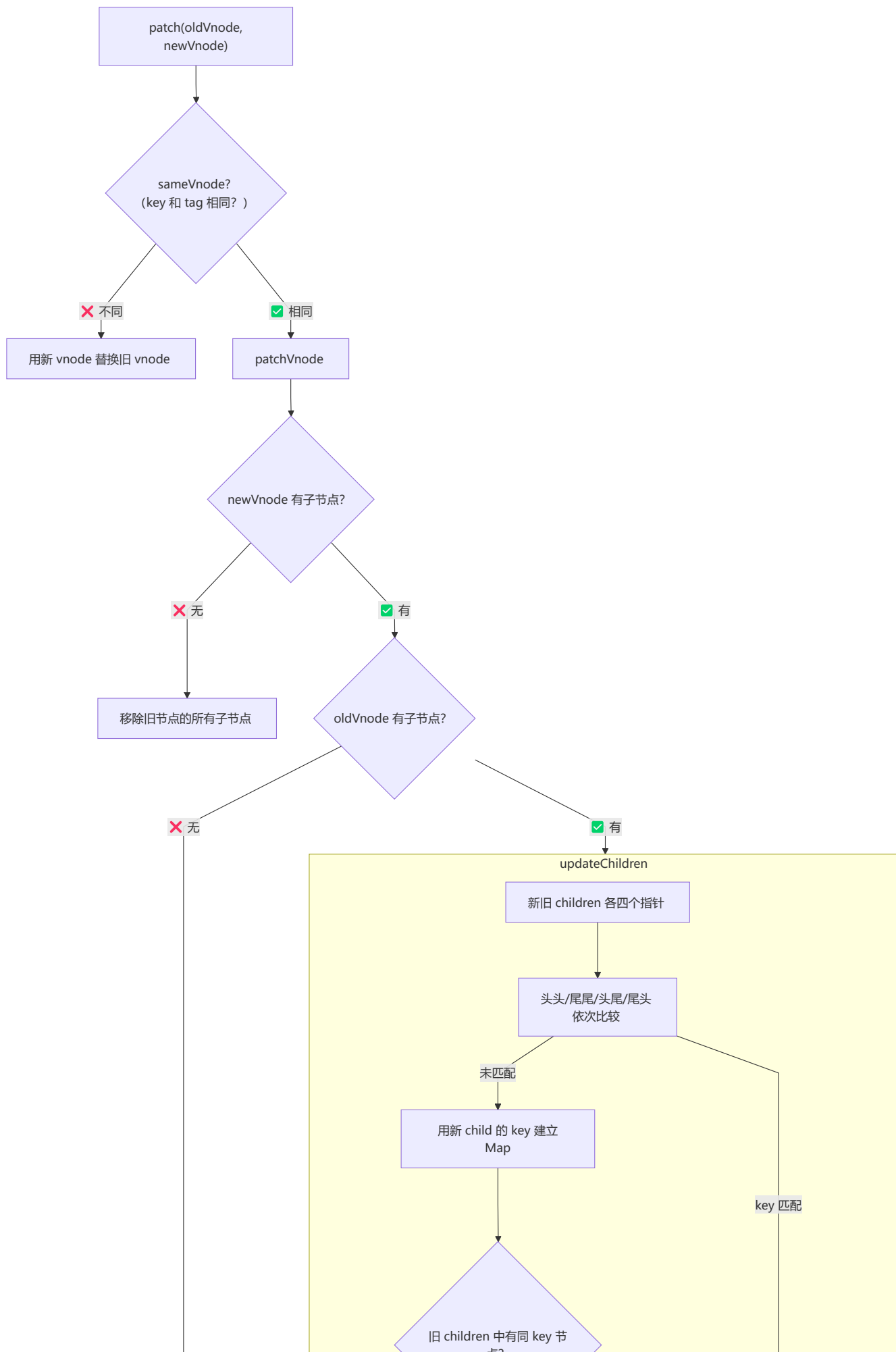


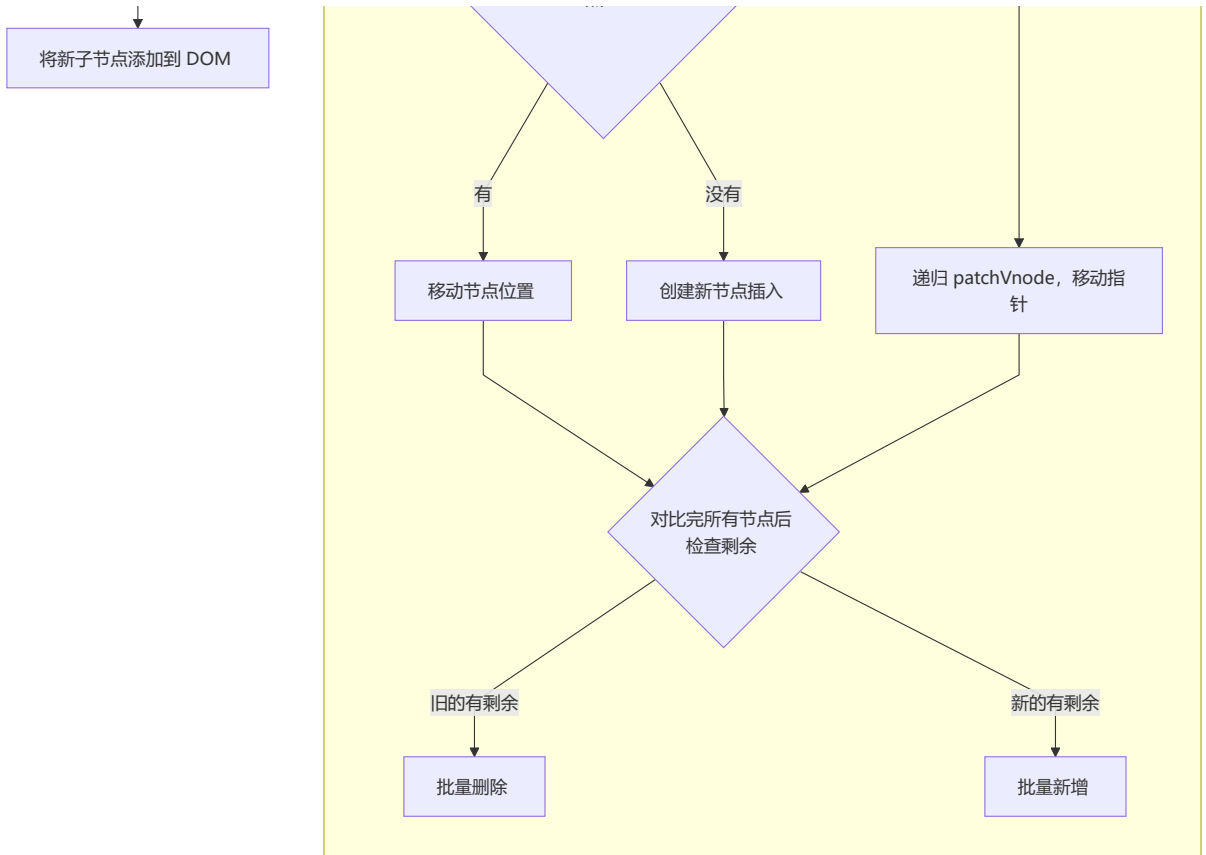
VNode 核心结构:

```
{
  tag: 'div',
  data: {
    attrs: { id: 'app' },
    class: ['foo'],
    style: { color: 'red' }
  },
  children: [
    { tag: 'span', text: 'hello' },
    { text: 'world' }
  ],
  elm: null,
  key: 'unique-id',
  componentOptions: {}
}
```

Diff 算法详解

Vue 的 Diff 算法采用**同层比较 + 四个指针遍历策略**，通过 key 建立 Map 快速查找可复用节点。



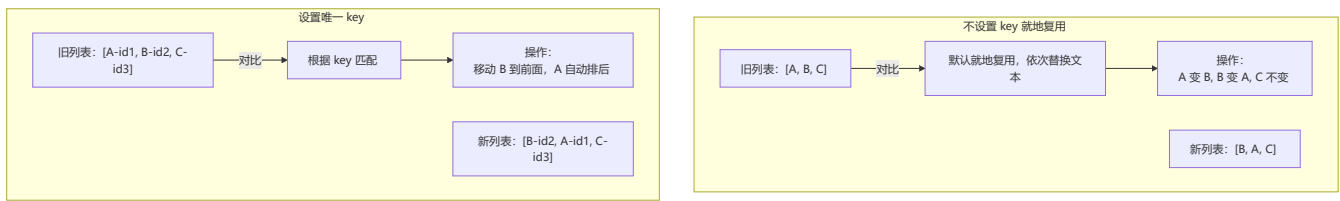


Diff 的关键优化：

1. **只做同层比较**：不跨层级比较 DOM 节点，时间复杂度从 $O(n^3)$ 降至 $O(n)$
2. **四个指针遍历**：头头、尾尾、头尾、尾头四组比较，提高匹配效率
3. **key 建立 Map**：通过 key 的 Map 快速查找可复用的节点

Key 的作用

key 帮助 Diff 算法准确识别节点身份，避免就地复用导致的渲染错误。**不要使用 index 作为 key。**

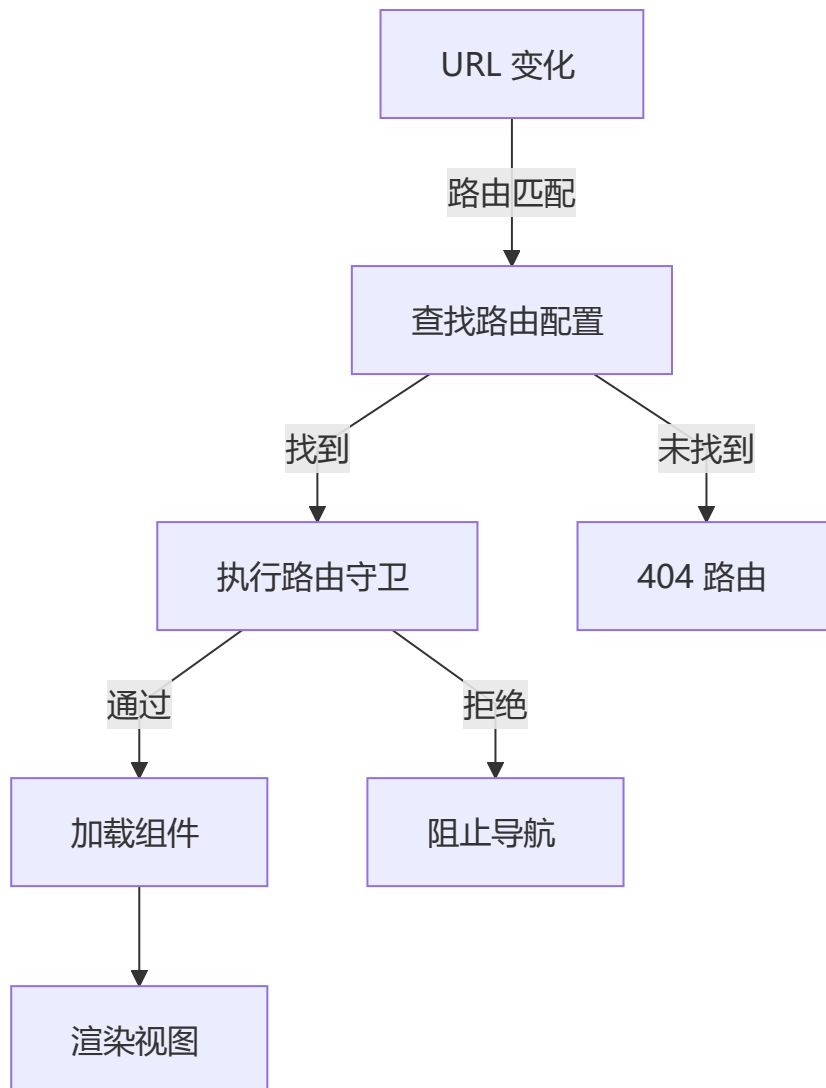


虚拟 DOM 的完整工作流程



10 路由系统详解

路由流程图



完整路由配置示例

```
// router/index.ts
import { createRouter, createWebHistory } from 'vue-router';
import type { RouteRecordRaw } from 'vue-router';

const routes: RouteRecordRaw[] = [
  {
    path: '/',
    name: 'home',
    component: () => import('../views/Home.vue')
  },
  {
    path: '/about',
    name: 'about',
    component: () => import('../views/About.vue')
  },
];
```

```

{
  path: '/user/:id',
  name: 'user-detail',
  component: () => import('../views/UserDetail.vue'),
  props: true
},
{
  path: '/dashboard',
  name: 'dashboard',
  component: () => import('../layouts/DashboardLayout.vue'),
  meta: { requiresAuth: true },
  children: [
    { path: '', name: 'dashboard-overview', component: () =>
import('../views/DashboardOverview.vue') },
    { path: 'settings', name: 'dashboard-settings', component: () =>
import('../views/DashboardSettings.vue') }
  ]
},
{
  path: '/*',
  name: 'not-found',
  component: () => import('../views/NotFound.vue')
}
];

const router = createRouter({
  history: createWebHistory(),
  routes
});

router.beforeEach((to, from, next) => {
  if (to.meta.requiresAuth && !isAuthenticated()) {
    next('/login');
  } else {
    next();
  }
});

export default router;

```

📌 在组件中使用路由

```

<script setup lang="ts">
import { useRoute, useRouter } from 'vue-router';
import { computed } from 'vue';

const route = useRoute();
const router = useRouter();

const userId = computed(() => route.params.id);
const searchQuery = computed(() => route.query.search);

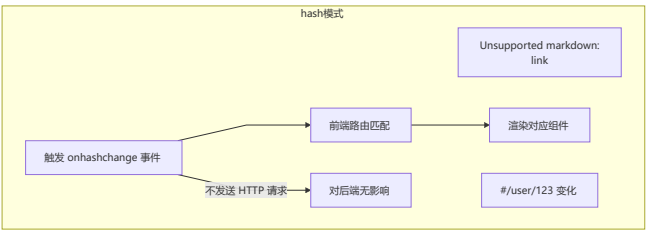
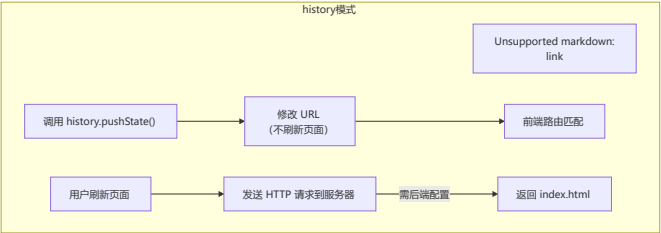
```

```
const goToHome = () => router.push('/');
</script>

<template>
  <router-link to="/">返回首页</router-link>
  <router-link :to="{ name: 'user-detail', params: { id: 456 } }">查看用户</router-link>
  <button @click="goToHome">首页</button>
  <router-view />
</template>
```

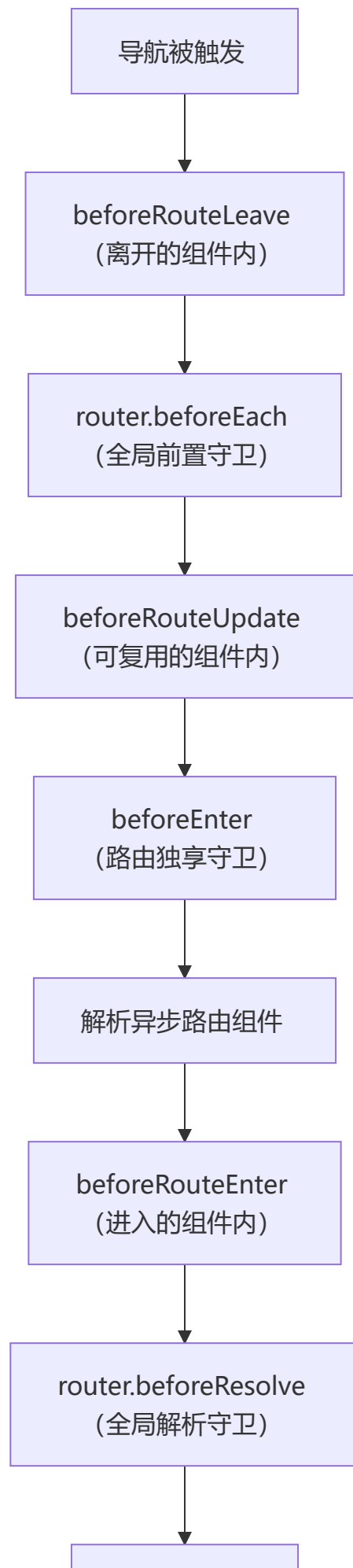
hash 模式 vs history 模式

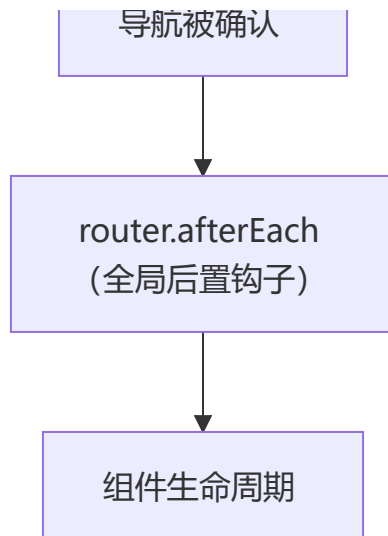
特性	hash 模式	history 模式
URL 格式	带 #	正常 URL
浏览器兼容性	IE8+	IE10+
后端配置	无需	需要
刷新行为	不触发请求	触发真实 HTTP 请求





导航守卫执行顺序





1 1 状态管理

Pinia vs Vuex

Pinia (推荐)	vuex (传统)
✓ 更简洁的 API	⚠ 样板代码多
✓ 完美的 TypeScript	⚠ 类型支持弱
✓ 无 Mutations	⚠ 必需 Mutations
✓ 按需导入	⚠ 需要导入全部
✓ 官方推荐	⚠ 逐步淘汰

Pinia 完整示例

```
// stores/counter.ts
import { defineStore } from 'pinia';
import { ref, computed } from 'vue';

// Setup 函数式 (推荐)
export const useCounterStore = defineStore('counter', () => {
  const count = ref(0);
  const name = ref('Counter Store');

  const doubled = computed(() => count.value * 2);
  const isPositive = computed(() => count.value > 0);

  const increment = () => count.value++;
  const decrement = () => count.value--;
  const reset = () => count.value = 0;

  const fetchRandomCount = async () => {
    const response = await fetch('https://api.example.com/random');
    const data = await response.json();
    count.value = data.number;
  };
});
```

```

    return { count, name, doubled, isPositive, increment, decrement, reset, fetchRandomCount
  };
});

// Options 式
export const useUserStore = defineStore('user', {
  state: () => ({ user: null, isLoading: false }),
  getters: {
    isLoggedIn: (state) => !!state.user,
    userName: (state) => state.user?.name || 'Guest'
  },
  actions: {
    async login(username: string, password: string) {
      this.isLoading = true;
      try {
        const response = await fetch('/api/login', { method: 'POST', body: JSON.stringify({
username, password }) });
        this.user = await response.json();
      } finally { this.isLoading = false; }
    },
    logout() { this.user = null; }
  }
});

```

Pinia State 操作方式:

```

const store = useCounterStore()

// 直接修改
store.count++

// $patch 批量修改（推荐多个属性同时修改）
store.$patch({ count: store.count + 1, name: 'updated' })

// $patch 函数式
store.$patch((state) => { state.count++; state.name = 'updated' })

// $reset 重置
store.$reset()

// 保持响应式解构
import { storeToRefs } from 'pinia'
const { count, name } = storeToRefs(store)

```

组件中使用 Pinia

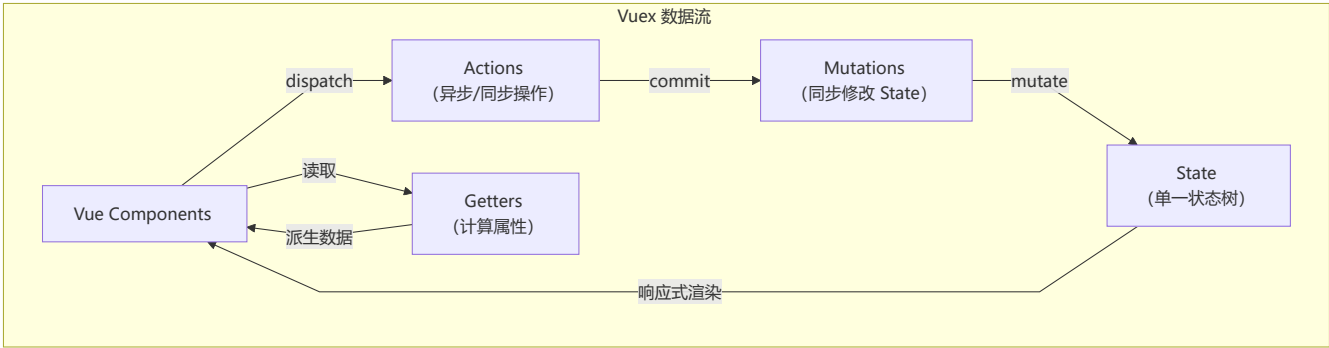
```
<script setup lang="ts">
import { useCounterStore } from '../stores/counter';
const counterStore = useCounterStore();
</script>

<template>
  <p>Count: {{ counterStore.count }}</p>
  <p>Doubled: {{ counterStore.doubled }}</p>
  <button @click="counterStore.increment">+1</button>
</template>
```



Vuex 核心架构

Vuex 采用单向数据流：Component → dispatch → Action → commit → Mutation → mutate → State → 响应式渲染到 Component。



五大核心属性：

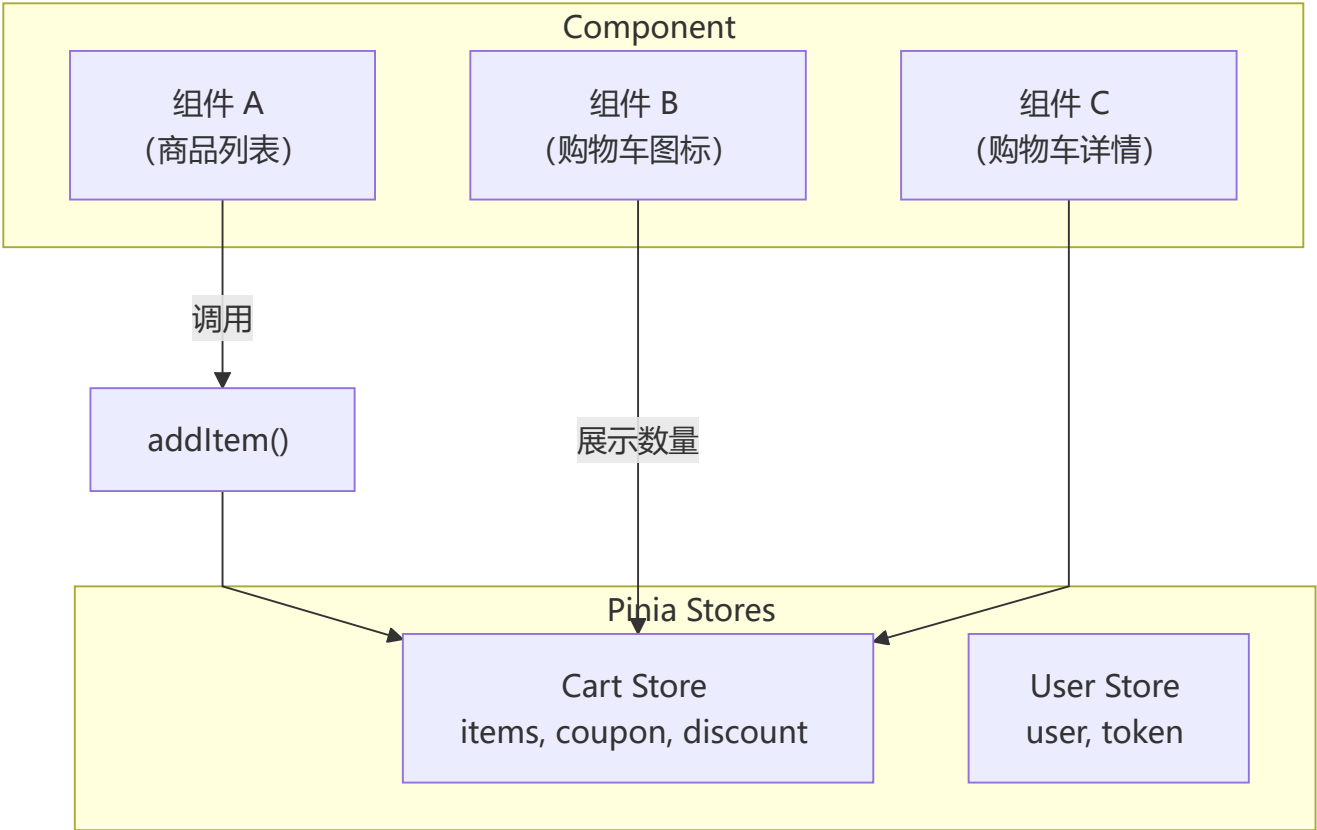
属性	作用	特点
State	数据源	单一状态树，响应式
Getter	派生数据	类似计算属性，有缓存
Mutation	同步修改 State	唯一修改方式，必须同步
Action	异步操作	提交 Mutation，可含异步逻辑
Module	模块化	拆分 Store，命名空间

Vuex vs Pinia 详细对比：

对比维度	Pinia	Vuex
API 设计	Composition API 风格，简洁直观	Options API 风格
TypeScript 支持	原生极佳，无需额外类型声明	需要复杂类型推导
模块化	无 Module 概念，每个 Store 独立	Module + 命名空间

对比维度	Pinia	Vuex
Mutation	✗ 无, 直接修改 State	✓ 必须通过 Mutation
代码体积	~1KB	~10KB
HMR	✓ 支持热更新	✗ 不支持

Pinia 数据流:



1 2 组合式 API 进阶

<script setup> 语法

```
<script setup>
import { ref, onMounted } from 'vue'

const count = ref(0)
const msg = ref('Hello')

onMounted(() => {
  console.log('mounted')
})
</script>

<template>
  <div>{{ count }} - {{ msg }}</div>
</template>
```

```
</template>
```

defineProps / defineEmits / defineExpose:

```
<script setup>
const props = defineProps({
  title: String,
  count: { type: Number, default: 0 }
})

const emit = defineEmits(['update', 'delete'])

function handleClick() {
  emit('update', props.count + 1)
}

defineExpose({ resetCount() { /* ... */ } })
</script>
```

defineModel (v-model 语法糖, Vue 3.4+) :

```
<!-- 子组件 -->
<script setup>
const model = defineModel({ type: String, default: '' })
</script>

<template>
  <input v-model="model" />
</template>

<!-- 父组件 -->
<ChildComponent v-model="value" />
```

defineOptions (Vue 3.3+) :

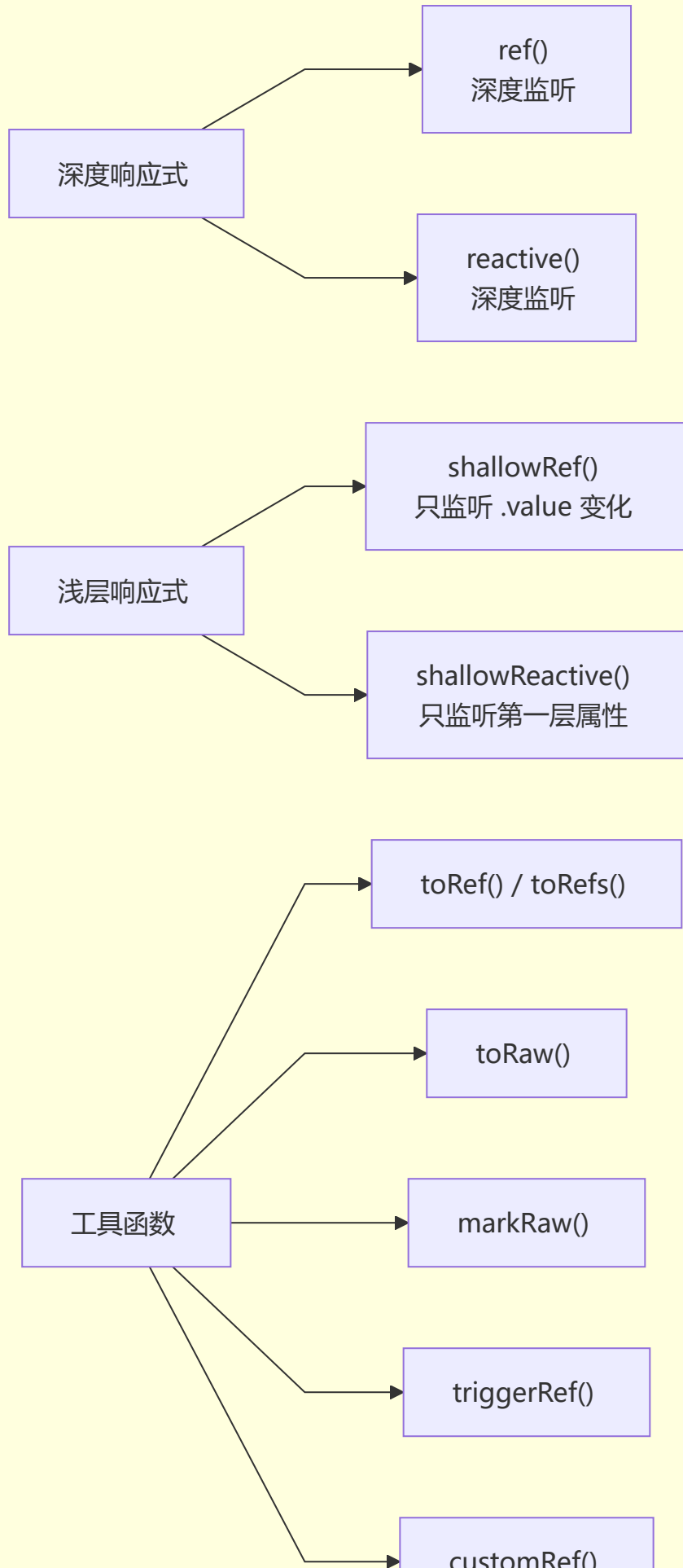
```
<script setup>
defineOptions({
  name: 'MyComponent',
  inheritAttrs: false
})
</script>
```

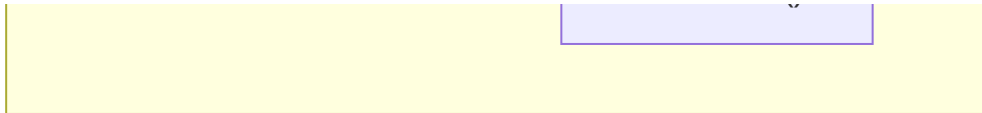
withDefaults:

```
<script setup>
interface Props {
  title?: string
  count?: number
  items?: string[]
}

const props = withDefaults(defineProps<Props>(), {
  title: '默认标题',
  count: 0,
  items: () => []
})
</script>
```


响应式分类





shallowRef / shallowReactive:

```
import { shallowRef, shallowReactive } from 'vue'

// shallowRef: 只追踪 .value 的变化，不深度监听内部属性
const state = shallowRef({ count: 0 })
state.value.count = 1 // ✗ 不触发响应式
state.value = { count: 1 } // ✔ 触发响应式

// shallowReactive: 只监听第一层属性的变化
const obj = shallowReactive({
  nested: { count: 0 }
})
obj.nested.count = 1 // ✗ 不触发响应式
```

triggerRef / customRef:

```
import { triggerRef, customRef, shallowRef } from 'vue'

// triggerRef: 强制触发依赖更新
const shallow = shallowRef({ count: 0 })
shallow.value.count = 1
triggerRef(shallow) // 手动触发更新

// customRef: 显式控制依赖收集和触发更新
function useDebounceRef(value, delay = 200) {
  let timeout
  return customRef((track, trigger) => ({
    get() {
      track()
      return value
    },
    set(newValue) {
      clearTimeout(timeout)
      timeout = setTimeout(() => {
        value = newValue
        trigger()
      }, delay)
    }
  })))
}
```

toRef / toRefs / toRaw / markRaw:

```
import { reactive, toRef, toRefs, toRaw, markRaw } from 'vue'

const state = reactive({ a: 1, b: 2 })
```

```
// toRef: 为 reactive 对象的单个属性创建 ref
const aRef = toRef(state, 'a')
aRef.value++ // state.a 也会改变

// toRefs: 将 reactive 对象的所有属性转为 ref
const { a, b } = toRefs(state)

// toRaw: 获取响应式对象的原始对象
const raw = toRaw(state)

// markRaw: 标记对象永远不会转为响应式
const heavyData = markRaw({ /* 大型不可变数据 */ })
```

effectScope:

```
import { effectScope, onScopeDispose, ref, computed, watch } from 'vue'

const scope = effectScope()

scope.run(() => {
  const count = ref(0)
  const double = computed(() => count.value * 2)

  watch(count, (newVal) => {
    console.log('count changed:', newVal)
  })

  onScopeDispose(() => {
    console.log('scope disposed')
  })
})

// 停止作用域: 所有 effect 被自动清理
scope.stop()
```

内置组件

`<Teleport>` (传送门) :

```
<template>
  <div class="component">
    <button @click="showModal = true">打开模态框</button>

    <!-- 模态框渲染到 body 下, 避免父组件 overflow:hidden 裁剪 -->
    <Teleport to="body">
      <div v-if="showModal" class="modal">
        <p>模态框内容</p>
      </div>
    </Teleport>

    <!-- 可禁用传送 -->
```

```

    <Teleport :disabled="!isMobile">
      <MobileNav />
    </Teleport>
  </div>
</template>

```

`<Suspense>` (异步依赖处理) :

```

<script setup>
import { defineAsyncComponent } from 'vue'

const AsyncComp = defineAsyncComponent(() => import('./HeavyComponent.vue'))
</script>

<template>
  <Suspense>
    <AsyncComp />
    <template #fallback>
      <div>加载中...</div>
    </template>
  </Suspense>
</template>

```

`<KeepAlive>` 进阶:

```

<script setup>
const tabs = ['Home', 'About', 'Contact']
const activeTab = ref('Home')

const isCached = (comp) => comp.name !== 'Contact'
</script>

<template>
  <KeepAlive :max="5" :include="isCached">
    <component :is="activeTab" />
  </KeepAlive>

  <!-- KeepAlive + Router -->
  <RouterView v-slot="{ Component, route }">
    <KeepAlive :include="route.meta.keepAlive ? [route.name] : []">
      <component :is="Component" :key="route.name" />
    </KeepAlive>
  </RouterView>
</template>

```

`<Transition>` / `<TransitionGroup>` 增强:

```

<template>
  <Transition name="fade"
    enter-from-class="opacity-0"
    enter-active-class="transition duration-500"

```

```

    leave-active-class="transition duration-300"
    @before-enter="onBeforeEnter"
    @enter="onEnter"
  >
    <div v-if="show">内容</div>
  </Transition>

  <TransitionGroup name="list" tag="ul">
    <li v-for="item in items" :key="item.id">{{ item.text }}</li>
  </TransitionGroup>
</template>

<style>
.list-enter-active, .list-leave-active { transition: all 0.5s ease; }
.list-enter-from, .list-leave-to { opacity: 0; transform: translateX(30px); }
.list-move { transition: transform 0.5s ease; }
</style>

```

函数式组件与渲染函数

h() 函数:

```

import { h, ref } from 'vue'

export default {
  setup() {
    const count = ref(0)
    return () => h('div', { class: 'counter' }, [
      h('span', `计数: ${count.value}`),
      h('button', { onClick: () => count.value++ }, '增加')
    ])
  }
}

```

渲染函数 + 指令/插槽:

```

import { h, withDirectives, resolveComponent } from 'vue'

export default {
  render() {
    const MyComp = resolveComponent('MyComponent')
    const vnode = h(MyComp, { title: 'Hello' }, {
      default: () => h('span', '默认插槽'),
      header: () => h('h1', '标题插槽')
    })
    return withDirectives(vnode, [ [vshow, this.visible] ])
  }
}

```

JSX/TSX 支持:

```
import { defineComponent, ref } from 'vue'

export default defineComponent({
  name: 'Button',
  props: { type: { type: String, default: 'primary' } },
  emits: ['click'],
  setup(props, { emit, slots }) {
    const count = ref(0)
    const handleClick = () => { count.value++; emit('click', count.value) }

    return () => (
      <button class={['btn', `btn-${props.type}`]} onClick={handleClick}>
        {slots.default?.() ?? '按钮'}
        {count.value > 0 && <span> (点击{count.value}次) </span>}
      </button>
    )
  }
})
```

可组合函数 (Composables)

常用 Composables 封装:

```
// composables/useFetch.ts
import { ref, onMounted } from 'vue'

export function useFetch<T>(url: string) {
  const data = ref<T | null>(null)
  const loading = ref(false)
  const error = ref<string | null>(null)

  const fetchData = async () => {
    loading.value = true
    error.value = null
    try {
      const response = await fetch(url)
      if (!response.ok) throw new Error(`HTTP ${response.status}`)
      data.value = await response.json()
    } catch (e) {
      error.value = (e as Error).message
    } finally { loading.value = false }
  }

  onMounted(() => fetchData())
  return { data, loading, error, fetch: fetchData }
}

// composables/useCounter.ts
import { ref, computed } from 'vue'

export function useCounter(initialValue = 0) {
  const count = ref(initialValue)
```

```

const doubled = computed(() => count.value * 2)
const increment = () => count.value++
const decrement = () => count.value--
const reset = () => count.value = initialValue
return { count, doubled, increment, decrement, reset }
}

```

在组件中使用：

```

<script setup lang="ts">
import { useFetch } from '../composables/useFetch'
import { useCounter } from '../composables/useCounter'

interface Post { id: number; title: string; body: string }

const { data: posts, loading, error } = useFetch<Post[]>(
  'https://jsonplaceholder.typicode.com/posts?_limit=5'
)
const { count, doubled, increment, decrement } = useCounter(0)
</script>

<template>
  <div>
    <h2>计数器: {{ count }} (Double: {{ doubled }})</h2>
    <button @click="increment">+</button>
    <button @click="decrement">-</button>

    <div v-if="loading">加载中...</div>
    <div v-if="error" class="error">{{ error }}</div>
    <ul v-if="posts">
      <li v-for="post in posts" :key="post.id">
        <strong>{{ post.title }}</strong>
      </li>
    </ul>
  </div>
</template>

```

Composables vs Mixins:

维度	Mixins	Composables
命名冲突	合并时可能冲突	解构可重命名，无冲突
来源清晰	不确定属性来自哪个 mixin	每次调用显式返回
参数传递	无法传递参数	支持接受参数
Tree-shaking	全部打包	按需导入

1 3 单文件组件最佳实践

现代单文件组件结构

```
<script setup lang="ts">
import { ref, computed, onMounted } from 'vue'

// 类型定义
interface User { id: number; name: string; email: string }

// Props 定义
const props = withDefaults(
  defineProps<{ title: string; user?: User; count?: number }>(),
  { count: 0 }
)

// Emits 定义
const emit = defineEmits<{ (e: 'update:user', user: User): void; (e: 'delete'): void }>()

// 状态
const loading = ref(false)
const error = ref<string | null>(null)

// 计算属性
const displayName = computed(() => props.user?.name || 'Unknown')

// 方法
const handleDelete = async () => {
  loading.value = true
  try { emit('delete') }
  catch (e) { error.value = (e as Error).message }
  finally { loading.value = false }
}

// 生命周期
onMounted(() => { console.log('Component mounted') })

defineExpose({ handleDelete })
</script>

<template>
  <div class="user-card">
    <h2>{{ props.title }}</h2>
    <p>Name: {{ displayName }}</p>
    <div v-if="loading" class="loading">加载中...</div>
    <button @click="handleDelete">删除</button>
  </div>
</template>

<style scoped>
.user-card { border: 1px solid #ccc; padding: 20px; border-radius: 8px; }
.loading { color: #1890ff; }
```

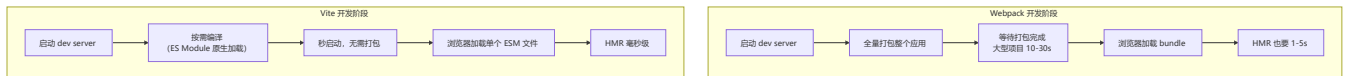


```
.error { color: #ff4d4f; }  
</style>
```

1 4 Vue 生态

🔥 Vite

Vite 基于原生 ES Module 实现秒级启动，开发时按需编译（esbuild），生产环境用 Rollup 打包；HMR 毫秒级，远快于 Webpack。



构建策略：

开发环境：esbuild（快，适用于 Dev）
↓
生产环境：Rollup（成熟，Tree-shaking 最佳）
↓
• 自动 CSS 代码分割
• 预加载指令生成
• 异步 chunk 加载
• 资源内联/优化

📦 Nuxt.js

Nuxt 提供 SSR/SSG/CSR 三种渲染模式，基于文件系统路由，自动导入 composables 和组件。



文件系统路由：

```
pages/  
├─ index.vue      → /  
├─ about.vue      → /about  
├─ blog/  
│   ├─ index.vue  → /blog  
│   └─ [id].vue   → /blog/:id  
└─ category/  
    └─ [...slug].vue → /category/* (catch-all)
```

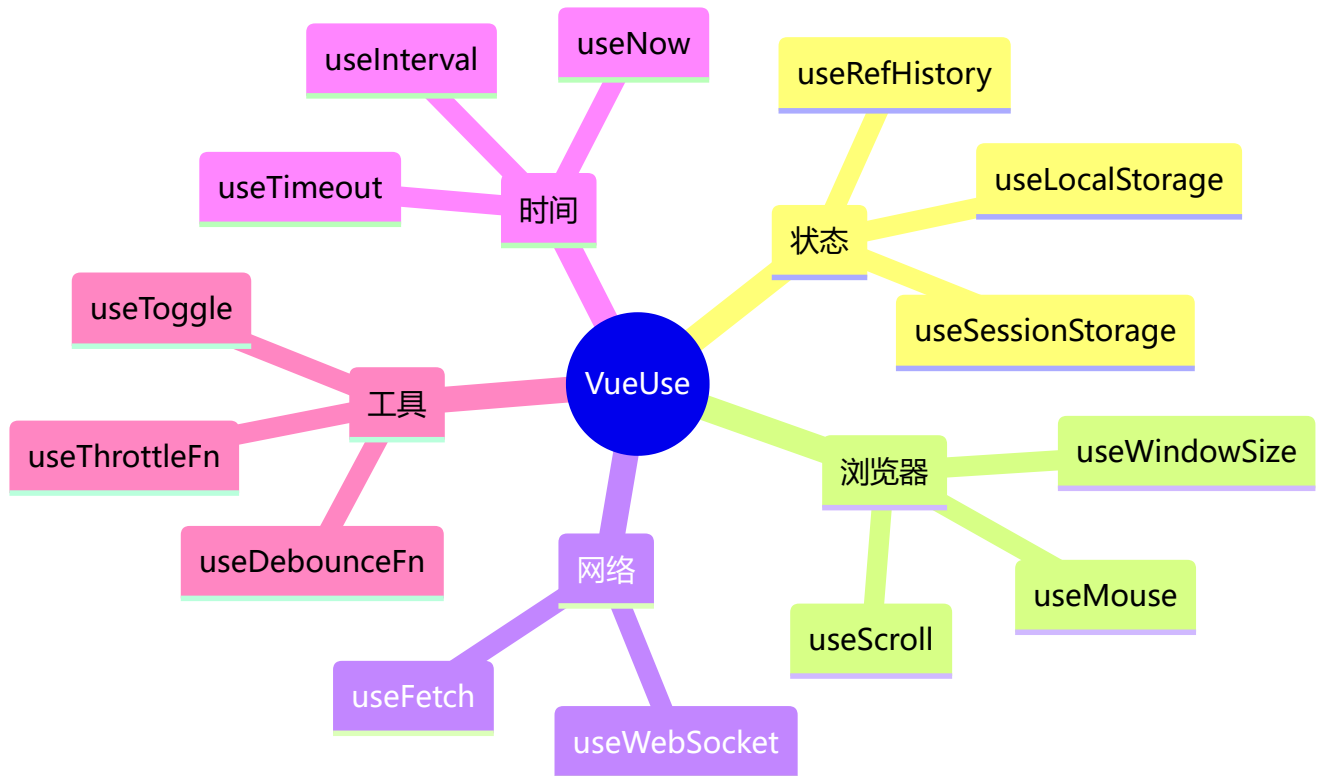
Nuxt vs Vue SPA 对比：

特性	Vue SPA (CSR)	Nuxt (SSR/SSG)
首屏白屏	❌ 存在	✅ 完整 HTML

特性	Vue SPA (CSR)	Nuxt (SSR/SSG)
SEO	✗ 差	✓ 优秀
部署	✓ 简单	需 Node 服务
适用场景	后台管理系统	官网/电商/博客

VueUse

基于 Composition API 的工具函数集合，提供 200+ 常用组合式函数。



常用函数示例：

```
import { useStorage, useMouse, useIntersectionObserver } from '@vueuse/core'

// 持久化状态（自动同步 localStorage）
const theme = useStorage('theme', 'light')
theme.value = 'dark' // 自动写入 localStorage

// 鼠标追踪
const { x, y } = useMouse()

// 元素可见性
const target = ref(null)
const { stop } = useIntersectionObserver(
  target,
  ([{ isIntersecting }]) => { targetIsVisible.value = isIntersecting },
  { threshold: 0.5 }
)
```

Vapor Mode

Vapor Mode 是 Vue 3 的一种可选编译策略，编译时跳过虚拟 DOM，直接生成高效的命令式 DOM 操作代码。

Vapor Mode

Template



编译为命令式 DOM 代码



直接操作 DOM



跳过 VNode 和 Diff

传统模式虚拟 DOM

Template

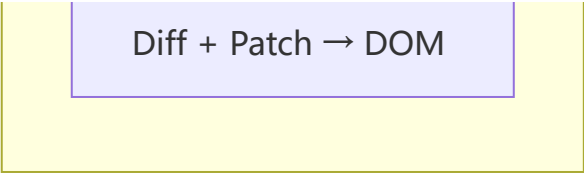


编译为 render 函数



运行时生成 VNode





特性	传统模式	Vapor Mode
虚拟 DOM	✅ 使用	❌ 跳过
运行时体积	~32KB	~6KB
内存占用	较高 (VNode 树)	更低
更新性能	依赖 Diff 算法	精确更新
灵活性	支持动态渲染、JSX	受限 (编译时优化)

第三部分：工程实践

1 5 工程化

Vite (Vue 3 推荐)

特性	Vite	Vue CLI (Webpack)
开发服务器	基于 ESM, 秒启动	全量打包, 慢
HMR	毫秒级	秒级
构建	Rollup	Webpack
创建项目	<code>npm create vue@latest</code>	<code>vue create my-app</code>

规范与约定

- **组件文件名**: PascalCase (如 `UserList.vue`)
- **Props 命名**: camelCase (JS) , kebab-case (模板)
- **CSS 类名**: kebab-case 或 BEM 规范
- **工具链**: ESLint + Prettier + Husky + Lint-staged

1 6 安全

XSS 防护

- Vue 模板默认对所有值进行 HTML 转义
- 谨慎使用 `v-html`，必要时使用 `DOMPurify` 净化
- 避免在客户端存储敏感数据（密码、API 密钥）

CSRF 防护

- 结合后端实现 CSRF token
- 在 `axios` 拦截器中自动添加
- 使用 `SameSite` 属性的 Cookie

HTTP 安全

- 全程 HTTPS 加密通信
- 配置 CSP（Content Security Policy）
- 合理配置 CORS 策略
- 依赖项审计：定期运行 `npm audit`

1 7 测试

单元测试

工具	用途
Vitest	基于 Vite 的单元测试框架，速度快
Vue Test Utils	挂载和与 Vue 组件交互的工具库

```
import { mount } from '@vue/test-utils';
import { describe, it, expect } from 'vitest';

describe('Counter Component', () => {
  it('should render with initial count 0', () => {
    const wrapper = mount(Counter);
    expect(wrapper.text()).toContain('Count: 0');
  });
});
```

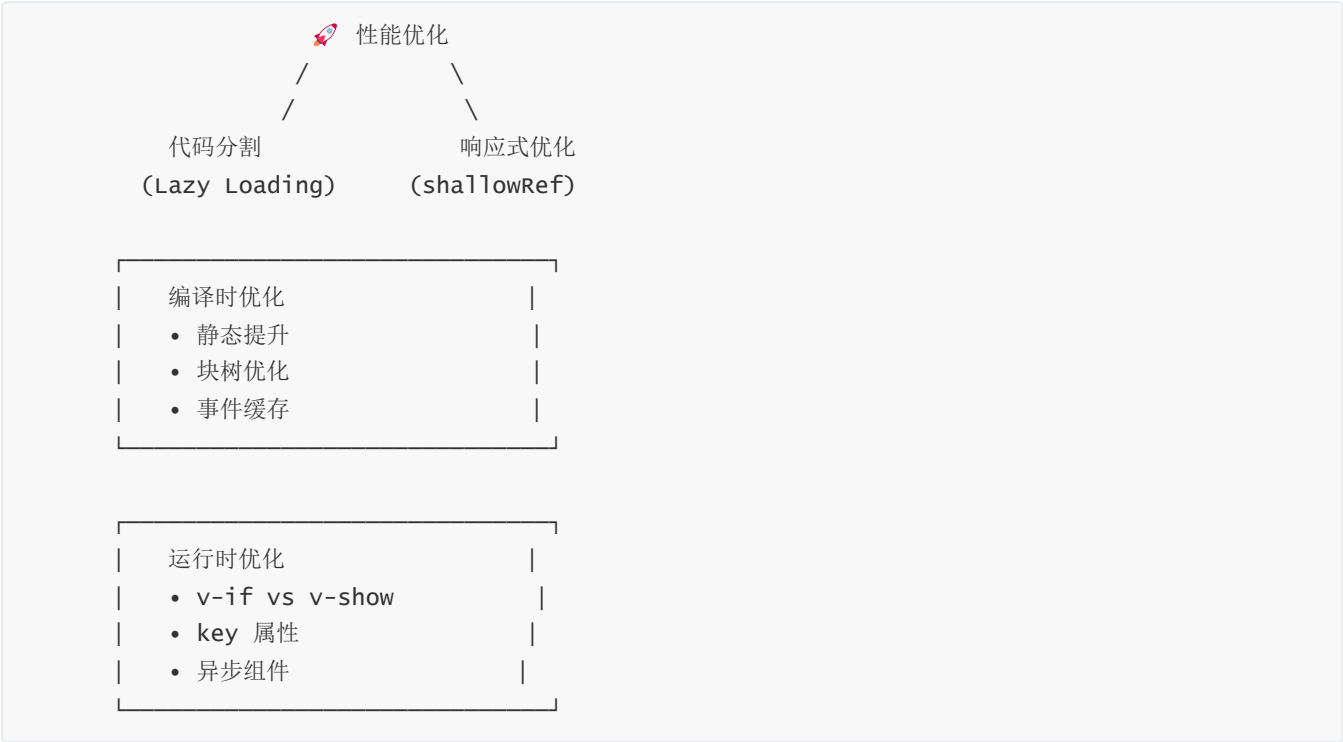
端到端测试

- Cypress：流行的 E2E 测试框架
- Playwright：微软的跨浏览器测试工具

第四部分：性能优化

1 8 性能优化完全指南

📦 优化策略金字塔



📌 常见优化技巧

合理使用 shallowRef

```
import { shallowRef, shallowReactive, triggerRef } from 'vue'

// ❌ 大型数据结构使用 ref 导致深度响应式
const data = ref({
  users: new Array(10000).fill({ /* ... */ }),
  config: { /* ... */ }
})

// ✅ 改为 shallowRef + triggerRef 手动控制
const data = shallowRef({
  users: new Array(10000).fill({ /* ... */ }),
  config: { /* ... */ }
})
```

```
function updateUsers(newUsers) {
  data.value = { ...data.value, users: newUsers }
  triggerRef(data) // 手动触发更新
}
```

v-once / v-memo

```
<template>
  <!-- v-once: 只渲染一次，后续不再更新 -->
  <div v-once>
    <h1>静态标题</h1>
    <p>创建时间: {{ createTime }}</p>
  </div>

  <!-- v-memo (Vue 3.2+)：依赖不变时不更新 -->
  <div v-for="item in list" :key="item.id" v-memo="[item.updated]">
    <!-- 只有 item.updated 变化时才重新渲染 -->
  </div>
</template>
```

函数式组件

```
// 函数式组件：无状态、无实例，渲染开销极低
import { h } from 'vue'

function StatelessButton(props, { slots, emit }) {
  return h('button', {
    class: `btn btn-${props.type}`,
    onClick: () => emit('click')
  }, slots.default?.())
}
```

异步组件 defineAsyncComponent

```
<script setup>
import { defineAsyncComponent, h } from 'vue'

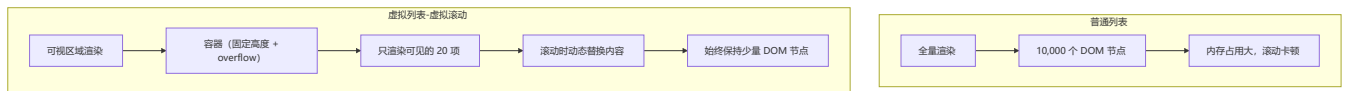
// 基础用法
const HeavyComp = defineAsyncComponent(() => import('./HeavyComponent.vue'))

// 完整配置
const AsyncComp = defineAsyncComponent({
  loader: () => import('./HeavyComponent.vue'),
  loadingComponent: () => h('div', '加载中...'),
  errorComponent: () => h('div', '加载失败'),
  delay: 200,
  timeout: 3000,
  suspense: true,
  onError(error, retry, fail, attempts) {
    if (attempts <= 3) retry()
    else fail()
  }
})
```



```
}  
})  
</script>
```

虚拟列表



其他优化要点

```
<!-- 优化 1: v-if vs v-show -->  
<Component v-if="isVisible" />      <!-- 初始快, 切换慢 -->  
<Component v-show="isVisible" />    <!-- 初始慢, 切换快 -->  
  
<!-- 优化 2: 正确使用 key -->  
<div v-for="item in items" :key="item.id">  
  {{ item.name }}  
</div>  
  
<!-- 优化 3: 异步组件懒加载 -->  
<script setup lang="ts">  
  const HeavyComponent = defineAsyncComponent(() => import('./HeavyComponent.vue'));  
</script>  
  
<!-- 优化 4: KeepAlive 缓存组件 -->  
<KeepAlive :max="10">  
  <component :is="currentComponent" />  
</KeepAlive>  
  
<!-- 优化 5: 浅层响应式 -->  
<script setup lang="ts">  
  import { shallowRef } from 'vue';  
  const items = shallowRef([ { id: 1, name: 'Item 1' } ]);  
  items.value = [...items.value, { id: 2, name: 'Item 2' }];  
</script>
```

编译时优化 (Vue 3 核心优势)

- **静态提升:** 编译器将从不改变的静态节点提升到渲染函数之外, 只创建一次
- **块树:** 只跟踪动态变化的块, 减少虚拟 DOM 比较开销
- **事件缓存:** 自动缓存内联事件处理器, 避免每次渲染创建新函数
- **Tree-shaking:** 移除未使用的 Vue 模块, 减小打包体积

第五部分：面试题汇总

核心概念面试题

Q1：Vue 3 相比 Vue 2 有什么主要改进？

标准答案：

1

性能提升

- 虚拟 DOM 重写，速度更快
- 静态提升和块树优化

2

响应式系统

- Proxy 替代 Object.defineProperty
- 支持更多数据类型的响应式

3

API 改进

- Composition API（组合式）
- 更好的 TypeScript 支持

4

开发体验

- 更小的包体积
- 更快的开发服务器（vite）

Q2：ref 和 reactive 的区别是什么？

特性	ref	reactive
适用类型	任何	仅对象
访问方式	.value	直接
模板解包	✓	✓
深层代理	✓	✓
支持重新赋值	✓	✗

何时使用：

- 简单值 → ref
- 复杂对象 → reactive 或 ref
- 需要重新赋值 → ref

Q3：组件通信有哪几种方式？

- 1

Props + \$emit（父子组件）
- 2

v-model（双向绑定）
- 3

provide/inject（祖先→后代）
- 4

Pinia（全局状态）
- 5

Event Bus / mitt（任意组件）
- 6

ref / \$refs（父调用子）

Q4：生命周期钩子的执行顺序是什么？

- 创建 → 挂载 → 更新 → 卸载
- 1

创建阶段：setup() / beforeCreate → created
- 2

挂载阶段：beforeMount → mounted
- 3

更新阶段：beforeUpdate → updated
- 4

卸载阶段：beforeUnmount → unmounted

Q5：为什么 Vue 3 使用 Proxy 代替 Object.defineProperty？

能力	Object.defineProperty (Vue 2)	Proxy (Vue 3)
拦截方式	逐个定义属性	代理整个对象
对象新增属性	✗ 需 Vue.set()	✓ 自动检测
对象删除属性	✗ 需 Vue.delete()	✓ 自动检测
数组下标修改	✗ 需重写方法	✓ 原生支持
数组长度变更	✗ 不可检测	✓ 原生支持
Map/Set	✗ 不支持	✓ 原生支持
初始化性能	递归遍历所有属性	懒代理（访问时才代理深层对象）

Q6：Composition API vs Options API

维度	Options API (mixin)	Composition API (composable)
命名冲突	mixin 合并时可能冲突	解构可重命名，无冲突
来源清晰	不确定属性来自哪个 mixin	每次调用显式返回
参数传递	无法传递参数	支持接受参数
多个实例	同一个 mixin 只能用一次	可多次调用
Tree-shaking	全部打包	按需导入
类型推导	困难	优秀

大型项目选择：

项目规模	推荐方案
小型组件 (< 50 行)	Options API
大型组件 (> 200 行)	Composition API
跨组件逻辑复用	Composition API (composable)

Q7: Pinia 相较于 Vuex 有哪些优势？

- **更少样板代码**：移除了 Mutations，直接在 Actions 中修改状态
- **完美的 TypeScript 支持**：默认提供强大的类型推断
- **模块化**：每个 Store 独立，无需嵌套 modules
- **轻量级**：~1KB，而 Vuex ~10KB
- **HMR 支持**：支持热更新

Q8: Vue 3 如何进行组件懒加载？

```
import { defineAsyncComponent } from 'vue'

// 基础用法
const HeavyComp = defineAsyncComponent(() => import('./HeavyComponent.vue'))

// 路由懒加载
const routes = [
  { path: '/', component: () => import('../views/Home.vue') }
]
```

`suspense` 用于处理异步组件加载时的 fallback 内容。

Q9: Vue 3 性能优化技巧有哪些？

1. **shallowRef**：大型数据避免深度响应式
2. **v-once / v-memo**：静态内容只渲染一次
3. **函数式组件**：无状态开销
4. **异步组件**：代码分割
5. **虚拟列表**：大量数据渲染
6. **合理使用 keep-alive**：缓存频繁切换的组件
7. **正确使用 key**：避免使用 index
8. **利用编译优化**：静态提升、块树、事件缓存

Q10: Vue 中如何处理安全性问题?

- **XSS**: 模板默认 HTML 转义, 谨慎使用 `v-html`, 使用 `DOMPurify` 净化
- **CSRF**: 结合后端 CSRF token, axios 拦截器添加
- **HTTPS**: 全程加密通信
- **CORS**: 合理配置跨域策略
- **输入验证**: 前后端双重验证
- **依赖审计**: 定期运行 `npm audit`

最佳实践总结

Vue 开发黄金法则

- 1 优先使用 Composition API (`<script setup>`)
→ 更易维护、易于复用, TypeScript 支持更好
- 2 充分利用 TypeScript
→ 类型安全、IDE 智能提示、减少运行时错误
- 3 为列表提供稳定的 key
→ 避免使用 index, 确保 Diff 算法高效复用
- 4 及时清理副作用
→ `onUnmounted` 中清理定时器、事件监听、订阅
- 5 合理选择状态管理方案
→ 小应用用 `Props + emit`, 大应用用 `Pinia`
- 6 利用 `computed` 缓存计算
→ 避免不必要的重复计算
- 7 组件按功能拆分
→ 单一职责原则, 易于维护和测试
- 8 充分利用 `slot` 进行组件组合
→ 增强组件的灵活性和可复用性
- 9 编写可复用的 `Composables`
→ 抽取业务逻辑为组合式函数, 提高复用性
- 10 使用 `shallowRef` 优化大型数据结构
→ 避免深度响应式监听的开销
- 1 1 合理选择 `v-if` 和 `v-show`
→ `v-if` 用于不频繁切换, `v-show` 用于频繁切换
- 1 2 异步组件实现代码分割
→ `defineAsyncComponent` + 路由懒加载

推荐学习资源

-  [官方文档](#)
-  [Vue 3 中文文档](#)
-  [Vue Mastery 教程](#)
-  [StackBlitz 在线编辑](#)
-  [Awesome Vue](#)

如有问题或建议，欢迎反馈！ 